

LiDAR-Inertial Odometry via Online RBF Terrain Surface Modeling for Legged-Wheel Robots

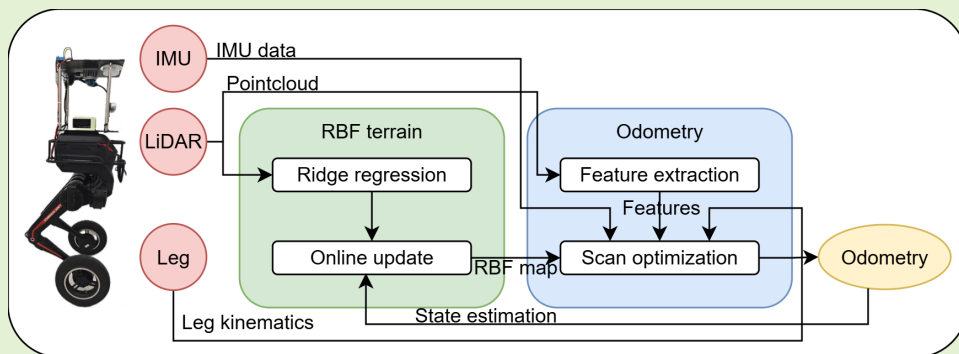
IEEE SENSORS JOURNAL, VOL. XX, NO. XX, XXXX 2024

1

Yizhe Liu, Chang Shen, Han Zhang, Jingchuan Wang, Weidong Chen

Abstract—Legged-wheel robots excel at traversing unstructured terrains, including bumpy roads and staircases. However, in such environments, neglecting terrain geometry can lead to severe odometry drifts. In particular, abrupt vertical accelerations caused by uneven surfaces may introduce significant pose errors along the z-axis, thereby degrading both localization and mapping accuracy. To address this issue, we propose a terrain-aware LiDAR-Inertial odometry (LIO) framework that approximates the terrain using Radial Basis Functions (RBFs) whose centers are adaptively selected and weights are recursively updated. The resulting smooth terrain manifold enables “soft constraints” that regularize the odometry optimization and mitigates the z-axis pose drift under abrupt elevation changes during robot’s maneuver. To ensure the LIO’s real-time performance, we further evaluate the RBF-related terms and calculate the inverse of the sparse kernel matrix with GPU parallelization. Experiments on unstructured terrains demonstrate that our method achieves higher localization accuracy than the baselines, especially in the scenarios that have continuous height changes or sparse features when abrupt height changes occur.

Index Terms—Simultaneous localization and mapping, terrain mapping, LiDAR-Inertial odometry(LIO), legged-wheel robot



I. INTRODUCTION

LEGGED-WHEEL robots combine the high mobility of wheeled platforms with the terrain adaptability of legged systems, making them well suited for inspection tasks in complex environments. In practice, they are often required to traverse challenging terrains such as staircases and bumpy roads. However, locomotion over such uneven terrain typically involves abrupt elevation changes, which may induce impulsive velocity variations. Such jolts pose significant challenges to odometry and often lead to pose estimation drift, particularly along the z -axis. This drift arises from the combined effect of inaccurate inertial propagation and insufficient geometric correction. In particular, on rough terrains, abrupt jolts and elevation changes make IMU-based propagation more susceptible to bias and noise accumulation, while LiDAR scan matching may not always provide sufficiently stable constraints on the robot pose. As a result, the corresponding estimation errors can gradually accumulate into noticeable

drift. This issue is particularly critical for legged-wheel robots because they are physically constrained by the terrain during locomotion. Therefore, terrain geometry naturally provides additional geometric constraints on the robot pose, especially on its height and attitude. If such geometry is not explicitly modeled and incorporated into the estimation framework, these constraints cannot be fully exploited to correct the accumulated errors, which in turn degrades both localization and mapping accuracy. Conventional techniques such as IMU deskewing, keyframing, and planar constraints do not fundamentally address this issue, since they reduce motion distortion, sparsify the optimization graph, or impose local flat-ground assumptions, but do not explicitly account for terrain geometry and the wheel-terrain contact relation. Moreover, explicit terrain modeling is also beneficial for downstream tasks such as terrain-aware motion planning and safe traversal over rough terrain. Earlier studies usually model the terrain using discrete 2.5-D elevation maps. Although such representations can capture the coarse elevation profile of the environment on a grid, they neglect fine-scale height variations within each cell. A smaller grid size may improve geometric fidelity, but 2.5-D elevation maps remain spatially discontinuous and non-differentiable. As a result, they are not well-suited for direct use in gradient-based optimization within odometry

Yizhe Liu and Chang Shen contributed equally to this work. All the authors are with School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Institute of Medical Robotics, Shanghai Jiao Tong University, and Key Laboratory of System Control and Information Processing, Ministry of Education, Shanghai 200240, China. Corresponding author: Han Zhang (e-mail: zhanghan_tc@sjtu.edu.cn)

frameworks, which motivates the need for a different terrain representation.

To this end, we use Radial Basis Functions (RBFs) to approximate the terrain. Compared with discrete 2.5-D elevation maps, the proposed representation provides a smooth terrain model directly reconstructed from LiDAR point clouds. Based on the moment conditions, we further develop a recursive ridge regression scheme in a Kalman-filter-like manner to update the RBF weights as new observations arrive. In this way, the terrain model can be continuously refined online as the robot moves. Since the resulting RBF-based terrain approximation defines a smooth manifold with explicitly available gradients, we further introduce terrain-aware soft constraints into the optimization of the LiDAR–Inertial Odometry (LIO) framework. These soft constraints augment the scan-matching objective with an additional terrain-consistency term, thereby improving the robustness of odometry. In particular, when abrupt height changes occur, the manifold constraint helps regularize the robot pose with respect to the approximated terrain surface and suppresses drift along the z -axis. In addition, to further improve real-time performance, both the evaluation of the RBF-related terms and the sparse kernel-matrix inversion are implemented using CUDA-based GPU kernels. To support accurate terrain reconstruction, the LiDAR is mounted in an inverted configuration (see Fig. ??) so as to enlarge its field of view over the ground surface. This design is motivated not only by the need for dense ground observations in the proposed terrain-aware odometry framework, but also by the requirement of reliable ground perception for downstream leg trajectory planning over uneven terrain.

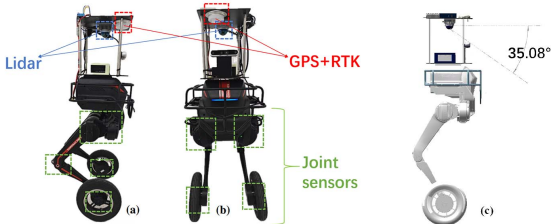


Fig. 1. Robot platform used in our experiments. An inverted Mid-360 LiDAR is mounted on the robot and used in all experiments. The platform is also equipped with a GPS+RTK module (red) and joint sensors (green) for kinematic state estimation. (a) side view, (b) front view, and (c) LiDAR field of view.

It is worth emphasizing that the novelty of this work does not lie in using RBF approximation, ridge regression, wheel–terrain contact, or LiDAR–inertial odometry as isolated components. Instead, the key contribution lies in coupling these components into a terrain-aware odometry framework in which the terrain is represented as an online-updated differentiable manifold and is directly used to regularize the pose optimization through wheel–terrain contact geometry. In this way, the terrain model is not only a mapping output or a preprocessing module, but also an active geometric constraint inside the LIO optimization. This design enables the estimator to exploit the physical contact between the wheel-legged platform and the terrain, which is particularly useful for suppressing accumulated vertical drift under abrupt elevation

changes.

To summarize, the main contributions of this work are three-fold:

- We propose an RBF-based terrain representation for uneven environments, which provides a smooth and differentiable approximation of the terrain from LiDAR point clouds. Based on moment conditions, a recursive ridge regression scheme is developed to update the RBF weights online as new observations arrive. In addition, an adaptive RBF center selection strategy and GPU parallelization are introduced to improve computational efficiency.
- We develop terrain-aware soft constraints that couple the online RBF terrain manifold with the wheel–terrain contact geometry and integrate them into the LIO scan-matching optimization. The proposed formulation uses the reconstructed terrain as an active geometric regularizer for pose estimation, thereby suppressing vertical drift and improving odometry robustness on rough terrain.
- We validate the proposed framework through extensive experiments on a legged-wheel robot platform in uneven-terrain scenarios. The proposed method outperforms FAST-LIO2 [?], GLIM [?], Leg-KILO [?], ROLO-SLAM [?] and Wheel-Legged SLAM [?] in challenging environments, especially in scenes with continuous height variations and sparse features. We also release the dataset¹ collected in this study to support future evaluation of SLAM methods for legged-wheel robots.

II. RELATED WORK

LiDAR–inertial odometry (LIO) has been widely studied in recent years. LOAM [?] achieves accurate real-time odometry and mapping using LiDAR-only input. Later works, such as LIO-SAM [?], Fast-LIO2 [?], [?], and GLIM [?], further improve robustness and accuracy by tightly coupling LiDAR and IMU measurements. However, these general-purpose LIO frameworks do not explicitly model terrain geometry, and therefore often suffer from pose drift when operating on height-varying or unstructured terrain.

To improve state estimation on legged or wheel-legged platforms, several works exploit kinematic or contact information. Leg-KILO [?] incorporates leg kinematics and extracts LiDAR points around the foot contact region to construct local contact-plane constraints. In particular, it assumes that the terrain around each foot contact is approximately flat within a radius of 10 cm. Wheel-Legged SLAM [?] similarly integrates the kinematic model of wheel-legged robots to improve LiDAR–inertial SLAM. VILENS [?] couples leg odometry with visual, inertial, and LiDAR measurements for quadruped robots, but it does not explicitly construct a continuous terrain model. LIKO [?] fuses LiDAR, IMU, and kinematic measurements from joint encoders and torque-related sensing on a biped robot, and estimates both the robot state and the foot contact position online. While these methods demonstrate the value of contact and kinematic constraints, they generally rely on local planar approximations, discrete contact events, or accurate contact

¹Available at https://zhanghan-tc.github.io/legged_wheel_dataset/

detection and non-slipping assumptions. However, they do not explicitly construct a continuous and differentiable terrain manifold for direct use in gradient-based LIO optimization. As a result, the associated terrain information is usually introduced only through local planar approximations or discrete contact events, rather than as a smooth function of the robot pose. Consequently, it is difficult for such constraints to provide consistent first-order geometric information throughout the iterative optimization process, which limits their ability to continuously correct height and attitude errors on uneven terrain. ROLO-SLAM [?] estimates the robot rotation first and then estimates the translation under the rotation constraint. However, for legged-wheel robots operating on uneven terrain, rotation and translation are often strongly coupled due to frequent body tilting. Consequently, such a decoupled strategy may lead to severe drift and degraded robustness.

Beyond contact- and kinematics-based methods, some SLAM systems have also been developed for legged robots from other perspectives. For example, ARS-SLAM [?] improves spinning-LiDAR SLAM for quadruped robots in large-scale scenarios, while FRL-SLAM [?] targets lightweight SLAM for quadruped navigation. These methods are relevant to legged-robot SLAM, but they do not focus on terrain-aware LiDAR-inertial estimation with an explicit continuous terrain model.

These limitations motivate the explicit use of terrain geometry in LIO design. To better capture the geometry of uneven environments, elevation mapping has been widely adopted. Early works on robot-centric elevation maps [?] and probabilistic terrain representations [?] improve terrain awareness by incorporating elevation uncertainty. Later methods exploit GPU acceleration for real-time elevation mapping [?]. Moreover, based on discrete elevation maps, [?] leverages terrain slope and ground constraints to improve vertical accuracy in LiDAR-inertial odometry. However, elevation maps are defined on fixed-resolution grids and are therefore not spatially differentiable. As a result, the corresponding constraints are typically derived from discretized grids or local planar patches, which yield only piecewise and non-smooth gradient information and are therefore less suitable for gradient-based optimization.

Beyond grid-based elevation maps, [?] employs Gaussian Process (GP) regression for probabilistic and continuous surface estimation. Although GP-based models provide smooth terrain representations, their computational cost typically scales poorly with the number of measurements, which makes real-time deployment challenging. On the other hand, radial basis functions (RBFs) offer an attractive alternative for surface reconstruction. In particular, RBFs provide continuous and dense terrain models, and have been successfully applied to 3D shape modeling [?] and digital terrain interpolation from airborne LiDAR data [?]. Therefore, RBF-based terrain approximation provides a continuous and differentiable representation that can be naturally integrated into optimization-based estimation. Nevertheless, as LiDAR measurements accumulate over time, direct least-squares estimation of the RBF weights becomes computationally expensive. Recursive approaches provide a systematic way to address this issue. In particular,

the connection between Recursive Least Squares (RLS) and the Kalman filter has been well established in optimization theory [?], enabling incremental updates of model parameters as new data arrive. However, this perspective remains largely underexplored in LIO systems.

Building on these observations, RBF-LIO uses an online RBF terrain function as a differentiable intermediate representation that connects terrain mapping and pose optimization through wheel-terrain contact geometry. The proposed method is related to several lines of prior work, but differs from them in how terrain geometry is represented, updated, and used in state estimation. General-purpose LIO methods such as LOAM, LIO-SAM, Fast-LIO2, and GLIM mainly rely on LiDAR geometric features and inertial propagation, but do not explicitly construct a terrain model that can constrain the robot height and attitude. Contact- or kinematics-aided methods such as Leg-KILO and Wheel-Legged SLAM exploit the robot morphology, but their terrain information is typically introduced through local flatness assumptions, discrete contact constraints, or kinematic consistency, rather than through a continuously differentiable terrain function. Elevation-map-based approaches can encode terrain height, but the grid representation makes the resulting gradients resolution-dependent and less suitable for direct smooth optimization. GP-based terrain models provide continuous surfaces, but their computational cost is less favorable for online LIO when measurements accumulate.

In contrast, RBF-LIO uses an RBF terrain function as an intermediate representation that connects terrain mapping and odometry optimization. The RBF model provides continuous height and gradient queries at arbitrary contact locations. The recursive ridge-regression update keeps the terrain representation online and computationally tractable, while the adaptive center selection avoids introducing unsupported or redundant kernels. More importantly, the approximated terrain manifold is coupled with wheel-terrain contact geometry and inserted into the scan-matching optimization as a soft constraint.

III. PRELIMINARIES AND PROBLEM FORMULATION

Before introducing the RBF-based terrain approximation, we first formulate the terrain-aware odometry problem considered in this work. This section clarifies why a continuous and differentiable terrain representation is required and how such a terrain model is used in the LiDAR-inertial odometry framework.

At the k -th LiDAR frame, let the robot base pose be denoted by

$$\eta_k = ({}^o\mathbf{R}_{B,k}, {}^o\mathbf{t}_{B,k}), \quad (1)$$

where ${}^o\mathbf{R}_{B,k} \in SO(3)$ and ${}^o\mathbf{t}_{B,k} \in \mathbb{R}^3$ denote the rotation matrix and translation vector from the base frame to the world frame, respectively. A standard feature-based LIO front-end estimates η_k by minimizing point-to-line and point-to-plane residuals extracted from the LiDAR scan. In this work, the

scan-matching objective is written as

$$J_{\text{scan}}(\boldsymbol{\eta}_k) = \sum_{\mathbf{p}_{s,k}^e \in \mathcal{F}_k^e} \|d_e(\boldsymbol{\eta}_k; \mathbf{p}_{s,k}^e)\|^2 + \sum_{\mathbf{p}_{s,k}^p \in \mathcal{F}_k^p} \|d_p(\boldsymbol{\eta}_k; \mathbf{p}_{s,k}^p)\|^2, \quad (2)$$

where $\mathcal{F}_k^e$ and $\mathcal{F}_k^p$ denote the extracted edge and planar feature sets. Moreover, $d_e(\cdot)$ and $d_p(\cdot)$ denote the corresponding point-to-line and point-to-plane residuals. This formulation is effective when sufficient and well-distributed LiDAR geometric features are available. However, for a legged-wheel robot moving on stairs, slopes and grass, the vertical direction can be weakly constrained by scan matching alone. Abrupt body motion over uneven terrain may also degrade inertial propagation, which further leads to accumulated drift along the z -axis.

A common way to describe terrain geometry is to use a discrete 2.5-D elevation map,

$$p_z = h_{\text{grid}}(p_x, p_y). \quad (3)$$

Although such a representation is convenient for terrain visualization and navigation, the height value is defined on discrete grid cells. Therefore, its gradient is usually obtained by finite differences and depends on the grid resolution. The resulting geometric constraint is only piecewise smooth and is not ideal for direct use in gradient-based LIO optimization. Another common simplification is to approximate the terrain around a contact point by a local plane,

$$\mathbf{n}^\top (\boldsymbol{\chi} - \mathbf{p}_0) = 0, \quad (4)$$

where $\boldsymbol{\chi}$ is the contact point, while $\mathbf{n}$ and $\mathbf{p}_0$ denote the local terrain normal and a point on the plane. This model is simple, but it only describes the terrain geometry in a small neighborhood. On stairs, curved slopes, or mixed grass-pathway regions, the local plane may change abruptly and cannot provide a consistent smooth terrain manifold for pose regularization.

These limitations motivate the use of a continuous terrain function

$$p_z = f(\mathbf{x}; \mathbf{w}), \quad \mathbf{x} = [p_x, p_y]^\top, \quad (5)$$

where $\mathbf{w}$ denotes the terrain model parameters. Such a terrain function should satisfy three requirements. First, it should predict the terrain height at arbitrary horizontal query points. Second, its spatial gradient should be available for constructing smooth residuals and Jacobians. Third, it should be updated online from streaming LiDAR measurements with bounded computational cost.

With this terrain representation, the wheel-terrain contact relation can be used as an additional geometric constraint for pose estimation. For each wheel $j \in \{L, R\}$, let $\boldsymbol{\chi}_{j,k}$ denote the corresponding contact point expressed in the world frame. Instead of enforcing the contact relation as a hard constraint, we define a soft manifold residual

$$\mathbf{r}_M^j = \mathbf{r}_M^j(\boldsymbol{\eta}_k, \boldsymbol{\chi}_{j,k}, f(\cdot; \mathbf{w})), \quad (6)$$

which measures the consistency among the robot pose, the wheel geometry, and the approximated terrain surface. The terrain-aware pose optimization considered in this work can then be summarized as

$$\min_{\boldsymbol{\eta}_k, \{\boldsymbol{\chi}_{j,k}\}} J_{\text{scan}}(\boldsymbol{\eta}_k) + \lambda_M \sum_{j \in \{L, R\}} \|\mathbf{r}_M^j(\boldsymbol{\eta}_k, \boldsymbol{\chi}_{j,k}, f(\cdot; \mathbf{w}))\|^2, \quad (7)$$

where λ_M is the weight of the manifold constraint.

In practice, the problem is solved online in two coupled steps. Given the current pose estimate and the local terrain point cloud, the RBF terrain model is first updated to obtain the smooth function $f(\mathbf{x}; \mathbf{w})$. Then, the resulting terrain manifold is queried at the wheel contact locations and introduced into scan matching through the residual $\mathbf{r}_M^j$.

IV. THE RBF-BASED TERRAIN APPROXIMATION

Based on the formulation in Sec. ??, this section introduces the RBF-based terrain approximation method. In particular, we first introduce the RBF approximation model for the terrain and then present the recursive ridge regression used for the weight update. Note that, in this section, the robot's base pose is assumed to be known.

A. The RBF Approximation Model

The RBF approximation is universal [?], i.e., given enough centers, it can approximate any continuous function on a compact set to arbitrary accuracy. Consequently, the RBF approximation model gives a dense, smooth representation for the terrain. Compared to 2.5-D elevation maps, the RBF approximation model can predict the height at any arbitrary point within the compact Region of Interest (ROI) $\mathcal{X}$. Moreover, its spatial gradient can be calculated explicitly at any point, which is essential for solving the optimization problem for the LIO.

To this end, we let the RBF approximation for the terrain be

$$f(\mathbf{x}; \mathbf{w}) = \sum_{i=1}^N w_i \phi(\|\mathbf{x} - \mathbf{c}_i\|_2), \quad (8)$$

where

- $\mathbf{x} = [p_x, p_y]^\top \in \mathbb{R}^2$ is a 2D query point in the horizontal plane, and $f(\mathbf{x}; \mathbf{w})$ denotes the predicted terrain height at the query point;
- $\mathbf{c}_{1:N} := \{\mathbf{c}_i = (x_i^c, y_i^c) \in \mathcal{X}\}_{i=1}^N \subset \mathbb{R}^2$ are the RBF center points, which will be selected adaptively based on the distribution of the terrain point cloud;
- $\mathbf{w} := \{w_i \in \mathbb{R}\}_{i=1}^N$ are the weights of the RBFs;
- $\phi(\cdot)$ is the RBF. In this work, we use the Gaussian kernel, namely,

$$\phi(r_i) = e^{-\frac{(r_i)^2}{2\sigma^2}} := \kappa_\sigma(\mathbf{x}; \mathbf{c}_j), \quad (9)$$

where $r_i = \|\mathbf{x} - \mathbf{c}_i\|_2$ is the Euclidean distance between the query point $\mathbf{x}$ and center $\mathbf{c}_i$, and $\sigma \in \mathbb{R}$ is a bandwidth parameter that determines the spatial influence of the kernel.

Notably, the model (??) predicts the terrain height as a continuous function of 2D spatial coordinates. The number

and placement of the RBF centers critically affect the approximation quality. In general, more centers improve the representation capability of the model. However, since the terrain is reconstructed from only a finite set of LiDAR measurements, using too many centers increases the computational cost and may also make the weight estimation numerically unstable. Moreover, due to the rapid decay of the Gaussian kernel, a center that is not sufficiently supported by nearby LiDAR points contributes little useful information to the regression. Hence, the number and placement of the centers must be carefully chosen to balance approximation accuracy, numerical stability, and computational efficiency.

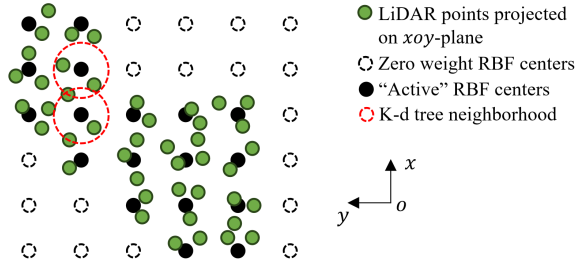


Fig. 2. The RBF center selection strategy illustration on xy -plane. Only candidate centers with sufficient nearby LiDAR support are retained.

To this end, we use an adaptive strategy to select the RBF centers, as illustrated in Fig. ?? . First, we build a k -d tree from the terrain point cloud for efficient neighborhood queries. Next, candidate centers are generated on a mesh with a specified resolution over the compact ROI $\mathcal{X}$ in the xy -plane. For each candidate center, the k -d tree counts the number of LiDAR points within a neighbourhood of radius r_{kd} . The candidate center is accepted only if this number exceeds a pre-defined threshold. In our implementation, we set $r_{kd} = \sigma$ so that the neighborhood test is consistent with the effective spatial support of the Gaussian kernel. Empirically, a minimum of three nearby LiDAR points is sufficient to yield reliable center selection. Such a strategy balances approximation accuracy, numerical stability, and computational efficiency.

B. The RBF Weight Estimate by Ridge Regression

To use (??) to approximate the terrain, we need to estimate the RBF weights. When no prior information about the terrain is available, e.g., at initialization, we estimate the RBF weights by solving a ridge regression problem. More specifically, since the robot's base pose is assumed to be known in this section, let $\{\mathbf{p}_k^i \in \mathbb{R}^3\}_{i=1}^{M_k}$, ($i = 1, \dots, M_k$), denote the LiDAR point cloud data at the k -th time step expressed in the world frame, where each point $\mathbf{p}_k^i := [x_k^i, p_{z,k}^i]^\top$ are approximately i.i.d. samples of the stochastic vector $\mathbf{p}$, and $\mathbf{x}_k^i := [p_{x,k}^i, p_{y,k}^i]^\top \in \mathbb{R}^2$. In addition, for the purpose of estimation, we model the observed LiDAR points $\mathbf{p}_k^i$, $\forall i, k$, expressed in the world frame, as samples of a stochastic vector $\mathbf{p}$. More specifically, we assume that

$$\mathbf{p} := \begin{bmatrix} \mathbf{x} \\ p_z \end{bmatrix} = \bar{\mathbf{p}} + \epsilon = \begin{bmatrix} \bar{\mathbf{x}} \\ \bar{p}_z \end{bmatrix} + \begin{bmatrix} \epsilon_x \\ \epsilon_z \end{bmatrix}, \quad (10)$$

where $\bar{\mathbf{p}}$ is a stochastic vector of 3D coordinates randomly drawn from the terrain surface expressed in the world frame, $\bar{\mathbf{x}} = [p_x, p_y]^\top$, and $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2 I)$ denotes the point cloud measurement noise, which is assumed to be independent of $\bar{\mathbf{p}}$. Further, suppose that (??) approximates the terrain surface well on $\mathcal{X}$, i.e.,

$$p_z = \underbrace{[\kappa_\sigma(\bar{\mathbf{x}}; \mathbf{c}_1) \cdots \kappa_\sigma(\bar{\mathbf{x}}; \mathbf{c}_N)]}_{\Phi(\bar{\mathbf{x}})^\top} \underbrace{\begin{bmatrix} w_1 \\ \vdots \\ w_N \end{bmatrix}}_{\mathbf{w}} + \epsilon_z, \quad (11)$$

where the centers $\mathbf{c}_j \in \mathcal{X}$, $j = 1, \dots, N$, and $\mathbf{w}$ is the vector of RBF weights.

Next, recall that our goal is to fit the terrain surface by estimating the RBF weights $\mathbf{w}$. However, we do not have access to i.i.d. samples of $\bar{\mathbf{x}}$, but only to the noisy observations $\mathbf{x}_k^i$ of $\mathbf{x}$. To this end, let $\mathbf{m}(\mathbf{x}) := \mathbb{E}[\Phi(\bar{\mathbf{x}}) | \mathbf{x}]$. Since ϵ_z is independent of $\mathbf{x}$, it holds that

$$\mathbb{E}[p_z | \mathbf{x}] = \mathbb{E}[\Phi(\bar{\mathbf{x}})^\top \mathbf{w} + \epsilon_z | \mathbf{x}] = \mathbb{E}[\Phi(\bar{\mathbf{x}})^\top | \mathbf{x}] \mathbf{w} = \mathbf{m}(\mathbf{x})^\top \mathbf{w}. \quad (12)$$

Therefore, by the tower property of conditional expectation, it follows that

$$\begin{aligned} \mathbb{E}[\mathbf{m}(\mathbf{x}) p_z] &= \mathbb{E}[\mathbb{E}[\mathbf{m}(\mathbf{x}) p_z | \mathbf{x}]] = \mathbb{E}[\mathbf{m}(\mathbf{x}) \mathbb{E}[p_z | \mathbf{x}]] \\ &= \mathbb{E}[\mathbf{m}(\mathbf{x}) \mathbf{m}(\mathbf{x})^\top] \mathbf{w}. \end{aligned} \quad (13)$$

Notably, (??) forms a moment equality, from which we obtain the estimator at the k -th time step:

$$\begin{aligned} \hat{\mathbf{w}}_k &= \mathbb{E}[\mathbf{m}(\mathbf{x}) \mathbf{m}(\mathbf{x})^\top]^{-1} \mathbb{E}[\mathbf{m}(\mathbf{x}) p_z] \\ &\approx \underbrace{\left(\sum_{t=1}^k \sum_{i=1}^{M_t} \mathbf{m}(\mathbf{x}_t^i) \mathbf{m}(\mathbf{x}_t^i)^\top \right)^{-1}}_{\mathbf{H}_k} \underbrace{\left(\sum_{t=1}^k \sum_{i=1}^{M_t} \mathbf{m}(\mathbf{x}_t^i) p_{z,t}^i \right)}_{\mathbf{b}_k}, \end{aligned} \quad (14)$$

where

$$\begin{aligned} \mathbf{m}(\mathbf{x}_t^i) &= \mathbb{E}[\Phi(\bar{\mathbf{x}}) | \mathbf{x}_t^i] \\ &= \frac{\sigma^2}{\tilde{\sigma}^2} [\kappa_{\tilde{\sigma}}(\mathbf{x}_t^i; \mathbf{c}_1) \cdots \kappa_{\tilde{\sigma}}(\mathbf{x}_t^i; \mathbf{c}_N)]^\top, \end{aligned} \quad (15)$$

with $\tilde{\sigma} = \sqrt{\sigma^2 + \sigma_\epsilon^2}$, since $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2 I)$. Moreover, (??) is the unique minimizer of

$$\min_{\mathbf{w}} \sum_{t=1}^k \sum_{i=1}^{M_t} (p_{z,t}^i - \mathbf{m}(\mathbf{x}_t^i)^\top \mathbf{w})^2. \quad (16)$$

However, $\mathbf{H}_k$ may be ill-conditioned due to the locality and overlap structure of the RBF kernels, which can significantly affect the approximation accuracy. To address this issue, we add a regularization term to (??) and solve a ridge regression problem [?], i.e.,

$$\min_{\mathbf{w}} \sum_{t=1}^k \sum_{i=1}^{M_t} (p_{z,t}^i - \mathbf{m}(\mathbf{x}_t^i)^\top \mathbf{w})^2 + \lambda \|\mathbf{w}\|^2, \quad (17)$$

and the corresponding estimator is

$$\hat{\mathbf{w}}_k^{RG} = \underbrace{(\mathbf{H}_k + \lambda I)}_{\mathcal{H}_k}^{-1} \mathbf{b}_k. \quad (18)$$

In particular, the ridge penalty $\lambda\|\mathbf{w}\|^2$ mitigates the ill-conditioning of $\mathbf{H}_k$ and numerically stabilizes the least-squares estimate in (??). Although this regularization introduces bias, the bias is typically modest in practice, while the reduction in variance may lead to a smaller mean-squared error [?]. The choice of λ will be further discussed in Sec. ??.

C. The Recursive RBF Weight Update

Recall the terrain approximation model (??). Since the RBF (??) decays exponentially to zero as r_i increases, the kernel $\kappa_\sigma(\mathbf{x}; \mathbf{c}_i)$ only captures the terrain geometry in the vicinity of the center $\mathbf{c}_i \in \mathcal{X}$. Therefore, at a new time step $k+1$, only those RBF centers that are sufficiently supported by the newly observed LiDAR points contribute significantly to the update. We refer to such centers as the *active RBF centers*, and the remaining ones as the *inactive RBF centers*.

According to (??), the updated estimator from the newly received LiDAR point cloud at time step $k+1$ takes the form

$$\hat{\mathbf{w}}_{k+1}^{RG} = (\mathbf{H}_{k+1})^{-1}(\mathbf{b}_k + \mathbf{m}_{k+1}\mathbf{p}_{z,k+1}), \quad (19a)$$

$$\mathbf{H}_{k+1} = \mathbf{H}_k + \mathbf{m}_{k+1}\mathbf{m}_{k+1}^\top, \quad (19b)$$

where $\mathbf{m}_{k+1} \in \mathbb{R}^{N \times M_{k+1}}$ and $\mathbf{p}_{z,k+1} \in \mathbb{R}^{M_{k+1}}$. More specifically,

$$\begin{aligned} \mathbf{m}_{k+1} &:= [\mathbf{m}(\mathbf{x}_{k+1}^1) \quad \cdots \quad \mathbf{m}(\mathbf{x}_{k+1}^{M_{k+1}})] \\ &= \frac{\sigma^2}{\bar{\sigma}^2} \begin{bmatrix} \kappa_{\bar{\sigma}}(\mathbf{x}_{k+1}^1; \mathbf{c}_1) & \cdots & \kappa_{\bar{\sigma}}(\mathbf{x}_{k+1}^{M_{k+1}}; \mathbf{c}_1) \\ \vdots & \ddots & \vdots \\ \kappa_{\bar{\sigma}}(\mathbf{x}_{k+1}^1; \mathbf{c}_N) & \cdots & \kappa_{\bar{\sigma}}(\mathbf{x}_{k+1}^{M_{k+1}}; \mathbf{c}_N) \end{bmatrix}, \end{aligned} \quad (20a)$$

$$\approx \frac{\sigma^2}{\bar{\sigma}^2} \begin{bmatrix} \mathbf{0} \\ \tilde{\mathbf{m}}_{k+1} \\ \mathbf{0} \end{bmatrix}, \quad (20b)$$

$$\mathbf{p}_{z,k+1} := [p_{z,k+1}^1 \quad \cdots \quad p_{z,k+1}^{M_{k+1}}]^\top. \quad (20c)$$

The approximation (??) follows from two facts: the LiDAR scans only cover a local portion of the ROI $\mathcal{X}$, and the kernel $\kappa_{\bar{\sigma}}(\mathbf{x}; \mathbf{c}_j)$ decays rapidly as the distance between $\mathbf{x}$ and $\mathbf{c}_j$ increases. Hence, most rows of $\mathbf{m}_{k+1}$ are numerically close to zero, except for those corresponding to the active RBF centers. We refer to these non-negligible rows as the *active rows*. After reordering the centers, the active rows can be grouped into a contiguous block, while the rows corresponding to inactive RBF centers remain approximately zero. Moreover, in view of (??), the update induced by $\mathbf{m}_{k+1}\mathbf{m}_{k+1}^\top$ only affects the sub-block associated with the active rows. Therefore, $\mathbf{H}_k$ exhibits a localized sparse structure, and the recursive update can be organized blockwise after reordering the centers. Furthermore, since $\mathbf{b}_k = \mathbf{H}_k \hat{\mathbf{w}}_k^{RG}$, (??) can be rewritten in a Kalman-filter-style form as

$$\begin{aligned} \hat{\mathbf{w}}_{k+1}^{RG} &= \mathbf{H}_{k+1}^{-1} \mathbf{b}_k + \mathbf{H}_{k+1}^{-1} \mathbf{m}_{k+1} \mathbf{p}_{z,k+1} \\ &= \mathbf{H}_{k+1}^{-1} \mathbf{H}_k \hat{\mathbf{w}}_k^{RG} + \mathbf{H}_{k+1}^{-1} \mathbf{m}_{k+1} \mathbf{p}_{z,k+1} \\ &= \mathbf{H}_{k+1}^{-1} (\mathbf{H}_{k+1} - \mathbf{m}_{k+1} \mathbf{m}_{k+1}^\top) \hat{\mathbf{w}}_k^{RG} + \mathbf{H}_{k+1}^{-1} \mathbf{m}_{k+1} \mathbf{p}_{z,k+1} \\ &= \hat{\mathbf{w}}_k^{RG} + \mathbf{H}_{k+1}^{-1} \mathbf{m}_{k+1} [\mathbf{p}_{z,k+1} - \mathbf{m}_{k+1}^\top \hat{\mathbf{w}}_k^{RG}]. \end{aligned} \quad (21)$$

Due to the sparse structure of $\mathbf{m}_{k+1}$ in (??), only the block associated with the active RBF centers needs to be updated. After reordering the centers, we partition $\hat{\mathbf{w}}_{k+1}^{RG}$ and $\mathbf{H}_{k+1}^{-1}$ conformably. Then (??) can be written as

$$\begin{aligned} \begin{bmatrix} \hat{\mathbf{w}}_{1,k+1}^{RG} \\ \hat{\mathbf{w}}_{2,k+1}^{RG} \\ \hat{\mathbf{w}}_{3,k+1}^{RG} \end{bmatrix} &= \begin{bmatrix} \hat{\mathbf{w}}_{1,k}^{RG} \\ \hat{\mathbf{w}}_{2,k}^{RG} \\ \hat{\mathbf{w}}_{3,k}^{RG} \end{bmatrix} + \begin{bmatrix} \mathbf{H}_{1,k+1}^{-1} & & \\ & \mathbf{H}_{2,k+1}^{-1} & \\ & & \mathbf{H}_{3,k+1}^{-1} \end{bmatrix} \begin{bmatrix} \mathbf{0} \\ \tilde{\mathbf{m}}_{k+1} \\ \mathbf{0} \end{bmatrix} \\ &\times \begin{bmatrix} \mathbf{p}_{z,k+1} - [\mathbf{0}^\top & \tilde{\mathbf{m}}_{k+1}^\top & \mathbf{0}^\top] \begin{bmatrix} \hat{\mathbf{w}}_{1,k}^{RG} \\ \hat{\mathbf{w}}_{2,k}^{RG} \\ \hat{\mathbf{w}}_{3,k}^{RG} \end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} \hat{\mathbf{w}}_{2,k}^{RG} + \mathbf{H}_{2,k+1}^{-1} \tilde{\mathbf{m}}_{k+1} (\mathbf{p}_{z,k+1} - \tilde{\mathbf{m}}_{k+1}^\top \hat{\mathbf{w}}_{2,k}^{RG}) \\ \hat{\mathbf{w}}_{3,k}^{RG} \end{bmatrix}. \end{aligned} \quad (22)$$

Moreover, by Woodbury matrix identity, it holds that

$$\begin{aligned} \mathbf{H}_{k+1}^{-1} &= \mathbf{H}_k^{-1} - \mathbf{H}_k^{-1} \mathbf{m}_{k+1} (I + \mathbf{m}_{k+1}^\top \mathbf{H}_k^{-1} \mathbf{m}_{k+1})^{-1} \mathbf{m}_{k+1}^\top \mathbf{H}_k^{-1} \\ &= \begin{bmatrix} \mathbf{H}_{1,k}^{-1} & & \\ & \mathbf{H}_{2,k+1}^{-1} & \\ & & \mathbf{H}_{3,k}^{-1} \end{bmatrix}, \end{aligned} \quad (23)$$

where

$$\mathbf{H}_{2,k+1}^{-1} := \mathbf{H}_{2,k}^{-1} - \mathbf{H}_{2,k}^{-1} \tilde{\mathbf{m}}_{k+1} (I + \tilde{\mathbf{m}}_{k+1}^\top \mathbf{H}_{2,k}^{-1} \tilde{\mathbf{m}}_{k+1})^{-1} \tilde{\mathbf{m}}_{k+1}^\top \mathbf{H}_{2,k}^{-1}. \quad (24)$$

In particular, the recursions (??) and (??) show that only the weights associated with the active RBF centers need to be updated, while the weights corresponding to the inactive RBF centers remain unchanged at the current step, as illustrated in Fig. ??.

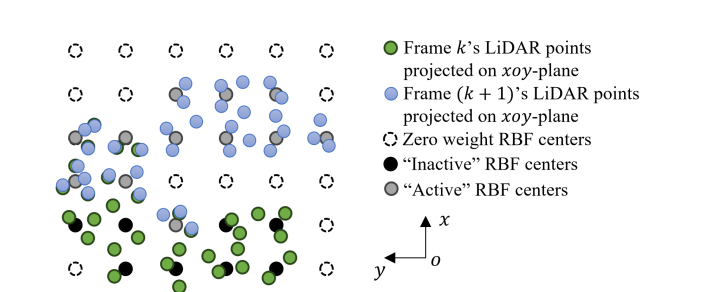


Fig. 3. Illustration of the recursive RBF weight update on the xoy -plane. Only the weights associated with the active RBF centers (gray) are updated, since they are supported by the newly received observations.

V. THE LIO WITH THE MANIFOLD CONSTRAINT

In previous section, we have obtained an approximation of the terrain. Next, we will use such an approximation as the “soft constraints” in the LIO system to improve the localization accuracy for a legged-wheel robot that maneuvers on the terrain. Such constraints can introduce additional geometric regularization when estimating the robot base pose, particularly in the vertical direction.

A. Leg Kinematics and Wheel–Terrain Contact

To exploit the manifold constraints, we first characterize the wheel–terrain contact geometry. To this end, let $\mathbf{q}_k$ denote the joint configuration at the time step k . For each wheel $j \in \{L, R\}$, we can easily calculate the wheel centers by the forward kinematics of the leg $\mathbf{h}_j(\mathbf{q})$, and hence the coordinate of wheel center ξ_j expressed in the world frame takes the form

$$\xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) := \begin{bmatrix} \xi_{x,j} \\ \xi_{y,j} \\ \xi_{z,j} \end{bmatrix} = {}^0\mathbf{R}_B \mathbf{h}_j(\mathbf{q}) + {}^0\mathbf{t}_B, \quad (25)$$

where ${}^0\mathbf{R}_B \in \text{SO}(3)$ and ${}^0\mathbf{t}_B \in \mathbb{R}^3$ denote the rotation matrix and translation vector that transform from the base frame to the world frame. Similarly, the unit vector along the wheel axis $\mathbf{u}_j$ in the world frame is given by

$$\mathbf{u}_j({}^0\mathbf{R}_B; \mathbf{q}) = {}^0\mathbf{R}_B \mathbf{g}_j(\mathbf{q}), \quad (26)$$

with $\mathbf{g}_j(\mathbf{q})$ being the axis directions in the base frame derived from leg kinematics.

Moreover, we assume that each wheel admits a unique effective contact point on the terrain at each time step. Let r be the wheel radius and $\chi_j = [\chi_{j,x}, \chi_{j,y}, \chi_{j,z}]^\top \in \mathbb{R}^3$ denote the contact point between the wheel and the terrain. This point must satisfy the following geometric contact constraints:

$$\text{On terrain:} \quad \chi_{j,z} - f(\chi_{j,x}, \chi_{j,y}) = 0, \quad (27a)$$

$$\text{In wheel plane:} \quad \mathbf{u}_j({}^0\mathbf{R}_B; \mathbf{q})^\top \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) = 0, \quad (27b)$$

$$\text{Radius condition:} \quad \|\chi_j - \xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})\|^2 - r^2 = 0, \quad (27c)$$

$$\text{Normal collinearity:} \quad \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) \times \mathbf{n}_g(\chi_j) = \mathbf{0}, \quad (27d)$$

where f is the RBF-based terrain height function defined in (??), $\mathbf{n}_g(\chi_j)$ is the terrain normal at the contact point computed from the gradient of f , $\mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})$ is the radial unit vector from the wheel center to the contact point. Specifically,

$$\mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) := \frac{\chi_j - \xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})}{r}, \quad (28a)$$

$$\mathbf{n}_g(\chi_j) := \frac{1}{\sqrt{1 + \left(\frac{\partial f}{\partial p_x}\right)^2 + \left(\frac{\partial f}{\partial p_y}\right)^2}} \begin{bmatrix} -\frac{\partial f}{\partial p_x}(\chi_{j,x}, \chi_{j,y}) \\ -\frac{\partial f}{\partial p_y}(\chi_{j,x}, \chi_{j,y}) \\ 1 \end{bmatrix}. \quad (28b)$$

We will use (??) as geometric consistency conditions later on in a soft-constrained optimization framework.

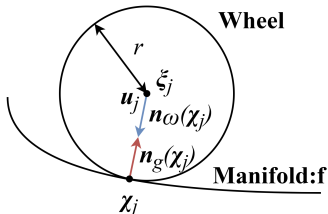


Fig. 4. Illustration of wheel–terrain contact.

B. IMU Pre-integration

Due to the relatively low frame rate of LiDAR scans, direct scan matching without a reliable initial guess is prone to convergence issues, especially under abrupt motions. To mitigate this issue, we use an IMU pre-integration module to integrate the high-rate inertial measurements between two consecutive LiDAR frames and provide a motion prior for scan matching.

To this end, let the state estimated by the LIO at time step k be

$$\eta_k = ({}^0\mathbf{R}_{B,k}, {}^0\mathbf{t}_{B,k}, {}^0\mathbf{v}_{B,k}, \mathbf{b}_k^a, \mathbf{b}_k^g), \quad (29)$$

where ${}^0\mathbf{R}_{B,k}$ and ${}^0\mathbf{t}_{B,k}$ denote the same pose variables as introduced in (??), ${}^0\mathbf{v}_{B,k} \in \mathbb{R}^3$ is the linear velocity expressed in the world frame, and $\mathbf{b}_k^a, \mathbf{b}_k^g \in \mathbb{R}^3$ are the accelerometer and gyroscope biases expressed in the local IMU frame, respectively.

Given discrete IMU measurements $(\mathbf{a}_i, \boldsymbol{\omega}_i)$ collected between the k -th and $(k+1)$ -th LiDAR scans, where $\mathbf{a}_i \in \mathbb{R}^3$ and $\boldsymbol{\omega}_i \in \mathbb{R}^3$ denote the linear acceleration and angular velocity measured at time t_i with sampling interval Δt_i , the pre-integrated relative motion can be computed recursively. Let the pre-integration terms be initialized as $\Delta \mathbf{R}_{k,k} = \mathbf{I}$, $\Delta \mathbf{v}_{k,k} = \mathbf{0}$, and $\Delta \mathbf{t}_{k,k} = \mathbf{0}$. Then the propagation from time t_i to t_{i+1} is given by

$$\Delta \mathbf{R}_{k,i+1} = \Delta \mathbf{R}_{k,i} \exp([\boldsymbol{\omega}_i - \mathbf{b}_k^g]_\times \Delta t_i), \quad (30a)$$

$$\Delta \mathbf{v}_{k,i+1} = \Delta \mathbf{v}_{k,i} + \Delta \mathbf{R}_{k,i} (\mathbf{a}_i - \mathbf{b}_k^a) \Delta t_i + \Delta t_i \mathbf{g}, \quad (30b)$$

$$\Delta \mathbf{t}_{k,i+1} = \Delta \mathbf{t}_{k,i} + \Delta \mathbf{v}_{k,i} \Delta t_i + \frac{1}{2} \Delta \mathbf{R}_{k,i} (\mathbf{a}_i - \mathbf{b}_k^a) \Delta t_i^2 + \frac{1}{2} \mathbf{g} \Delta t_i^2. \quad (30c)$$

Here, $\mathbf{g}$ denotes the gravity vector in world frame, $\exp(\cdot)$ denotes the matrix exponential on the Lie algebra $\mathfrak{so}(3)$, and $[\cdot]_\times$ maps a vector in $\mathbb{R}^3$ to its corresponding skew-symmetric matrix in $\mathfrak{so}(3)$. The pre-integrated quantities over the interval $[t_k, t_{k+1}]$ are obtained when $t_i = t_{k+1}$, and are then used to initialize the scan-matching optimization in the LIO framework.

C. The Manifold Constraints

We assume that the wheels maintain firm contact with the terrain. Accordingly, for each wheel $j \in \{L, R\}$, we define a manifold residual vector by stacking the three scalar contact constraints together with the vector-valued normal-collinearity residual:

$$\mathbf{r}_{\mathcal{M}}^j(\eta_k, \chi_j) := \begin{bmatrix} r_{j,1} \\ r_{j,2} \\ r_{j,3} \\ r_{j,4} \end{bmatrix} = \begin{bmatrix} \chi_{j,z} - f(\chi_{j,x}, \chi_{j,y}) \\ \mathbf{u}_j({}^0\mathbf{R}_B; \mathbf{q})^\top \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) \\ \|\chi_j - \xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})\|^2 - r^2 \\ \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) \times \mathbf{n}_g(\chi_j) \end{bmatrix} \approx \mathbf{0}. \quad (31)$$

Next, we incorporate the manifold residuals into the scan-matching optimization. Our LIO framework follows the front-end structure of LIO-SAM [?]. In particular, the IMU pre-integration module introduced in Sec. ?? is used to provide motion priors between consecutive LiDAR frames, and the

associated factor graph is used to update the accelerometer and gyroscope biases.

The scan matching adopted in this work follows the feature-based ICP formulation in LOAM [?] and LIO-SAM [?]. To initialize the ICP optimization, we use the IMU pre-integration between two consecutive frames to estimate the relative motion, which is then composed with the global pose of the previous frame to generate an initial guess for the current frame. More specifically, the scan-matching cost is given by

$$J_{\text{scan}}(\eta_k) = \sum_{\mathbf{p}_{s,k}^e \in \mathcal{F}_k^e} \left\| \mathbf{d}_e({}^O\mathbf{R}_{B,k}, {}^O\mathbf{t}_{B,k}; \mathbf{p}_{s,k}^e) \right\|^2 + \sum_{\mathbf{p}_{s,k}^p \in \mathcal{F}_k^p} \left\| \mathbf{d}_p({}^O\mathbf{R}_{B,k}, {}^O\mathbf{t}_{B,k}; \mathbf{p}_{s,k}^p) \right\|^2, \quad (32)$$

with respect to ${}^O\mathbf{R}_{B,k}$ and ${}^O\mathbf{t}_{B,k}$, where $\mathcal{F}_k^e$ and $\mathcal{F}_k^p$ denote the sets of edge and planar features in the LiDAR scan at time step k . Here, $\mathbf{p}_{s,k}^e$ and $\mathbf{p}_{s,k}^p$ denote the s -th edge and planar feature points extracted from the scan, while $\mathbf{d}_e(\cdot)$ and $\mathbf{d}_p(\cdot)$ are the corresponding point-to-line and point-to-plane residuals.

We then incorporate the manifold residuals into the scan-matching objective as soft constraints for pose regularization. The corresponding manifold cost is defined as

$$J_{\mathcal{M}}(\eta_k, \chi_j) = \sum_{j \in \{\text{L}, \text{R}\}} \left\| \mathbf{r}_{\mathcal{M}}^j(\eta_k, \chi_j) \right\|^2, \quad (33)$$

where the contact points χ_j 's are jointly optimized with the pose variables η_k in the manifold-constraint term. Therefore, the total cost function takes the form

$$J_{\text{total}}(\eta_k, \chi_j) = J_{\text{scan}}(\eta_k) + J_{\mathcal{M}}(\eta_k, \chi_j). \quad (34)$$

In particular, we minimize (??) using the Levenberg-Marquardt algorithm [?]. Since the Jacobians of the point-to-line and point-to-plane residuals have been discussed in detail in [?], we only derive the Jacobian of the newly introduced manifold residual $\mathbf{r}_{\mathcal{M}}^j(\eta_k, \chi_j)$ here. In the Levenberg-Marquardt solver, this residual is linearized around the current estimate with respect to the local pose increment and the contact-point increment. Accordingly, for each wheel, we define the joint increment vector as

$$\delta \mathbf{x}_j = \begin{bmatrix} \delta \eta \\ \delta \chi_j \end{bmatrix}, \quad \delta \eta = \begin{bmatrix} \delta \theta \\ \delta \mathbf{t} \end{bmatrix},$$

where $\delta \eta$ denotes the local increment of the base pose. The corresponding Jacobian takes the block form

$$\mathbf{J}_j = \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \mathbf{x}_j} = \begin{bmatrix} \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \theta} & \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \mathbf{t}} & \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \chi_j} \end{bmatrix}. \quad (35)$$

Using a right-perturbation model for the rotation increment $\delta \theta$, the derivatives of the wheel center and wheel-axis direction with respect to the pose increments are

$$\begin{aligned} \frac{\partial \xi_j}{\partial \delta \theta} &= -{}^O\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, & \frac{\partial \xi_j}{\partial \delta \mathbf{t}} &= \mathbf{I}_3, \\ \frac{\partial \mathbf{u}_j}{\partial \delta \theta} &= -[\mathbf{u}_j]_{\times} {}^O\mathbf{R}_B, & \frac{\partial \mathbf{u}_j}{\partial \delta \mathbf{t}} &= \mathbf{0}. \end{aligned} \quad (36)$$

With these preliminaries, we derive the Jacobian blocks for each residual component in (??). Let $\mathbf{d}_j := \chi_j - \xi_j$. It follows that

- **On terrain:** $r_{j,1} = \chi_{j,z} - f(\chi_{j,x}, \chi_{j,y})$. Its Jacobian takes the form

$$\frac{\partial r_{j,1}}{\partial \delta \theta} = \mathbf{0}_{1 \times 3}, \quad \frac{\partial r_{j,1}}{\partial \delta \mathbf{t}} = \mathbf{0}_{1 \times 3}, \quad \frac{\partial r_{j,1}}{\partial \delta \chi_j} = \begin{bmatrix} -\frac{\partial f}{\partial x} & -\frac{\partial f}{\partial y} & 1 \end{bmatrix}. \quad (37)$$

- **In wheel plane:** $r_{j,2} = \mathbf{u}_j({}^O\mathbf{R}_B; \mathbf{q})^\top \mathbf{n}_\omega(\chi_j, {}^O\mathbf{R}_B, {}^O\mathbf{t}_B; \mathbf{q})$. Since $\mathbf{n}_\omega = \mathbf{d}_j/r$, it follows that

$$\begin{aligned} \frac{\partial r_{j,2}}{\partial \delta \theta} &= \frac{1}{r} (\mathbf{u}_j^\top {}^O\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times} - \mathbf{d}_j^\top [\mathbf{u}_j]_{\times} {}^O\mathbf{R}_B), \\ \frac{\partial r_{j,2}}{\partial \delta \mathbf{t}} &= -\frac{\mathbf{u}_j^\top}{r}, & \frac{\partial r_{j,2}}{\partial \delta \chi_j} &= \frac{\mathbf{u}_j^\top}{r}. \end{aligned} \quad (38)$$

- **Radius condition:** $r_{j,3} = \|\chi_j - \xi_j({}^O\mathbf{R}_B, {}^O\mathbf{t}_B; \mathbf{q})\|^2 - r^2$. Its Jacobian takes the form

$$\begin{aligned} \frac{\partial r_{j,3}}{\partial \delta \theta} &= 2 \mathbf{d}_j^\top {}^O\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, \\ \frac{\partial r_{j,3}}{\partial \delta \mathbf{t}} &= -2 \mathbf{d}_j^\top, & \frac{\partial r_{j,3}}{\partial \delta \chi_j} &= 2 \mathbf{d}_j^\top. \end{aligned} \quad (39)$$

- **Normal collinearity:** $\mathbf{r}_{j,4} = \mathbf{n}_\omega(\chi_j, {}^O\mathbf{R}_B, {}^O\mathbf{t}_B; \mathbf{q}) \times \mathbf{n}_g(\chi_j)$. First, the derivatives of $\mathbf{n}_\omega$ take the form

$$\frac{\partial \mathbf{n}_\omega}{\partial \delta \theta} = \frac{1}{r} {}^O\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, \quad \frac{\partial \mathbf{n}_\omega}{\partial \delta \mathbf{t}} = -\frac{1}{r} \mathbf{I}_3, \quad \frac{\partial \mathbf{n}_\omega}{\partial \delta \chi_j} = \frac{1}{r} \mathbf{I}_3. \quad (40)$$

On the other hand, let

$$\mathbf{g}_f := \begin{bmatrix} -\frac{\partial f}{\partial p_x}(\chi_{j,x}, \chi_{j,y}) \\ -\frac{\partial f}{\partial p_y}(\chi_{j,x}, \chi_{j,y}) \\ 1 \end{bmatrix}, \quad \mathbf{H}_f := \begin{bmatrix} -\frac{\partial^2 f}{\partial p_x^2} & -\frac{\partial^2 f}{\partial p_x \partial p_y} & 0 \\ -\frac{\partial^2 f}{\partial p_y \partial p_x} & -\frac{\partial^2 f}{\partial p_y^2} & 0 \\ 0 & 0 & 0 \end{bmatrix}. \quad (41)$$

Then it holds that $\frac{\partial \mathbf{n}_g}{\partial \delta \chi_j} = \frac{1}{\|\mathbf{g}_f\|_2} (\mathbf{I}_3 - \mathbf{n}_g \mathbf{n}_g^\top) \mathbf{H}_f$. Applying the product rule for cross products yields

$$\begin{aligned} \frac{\partial \mathbf{r}_{j,4}}{\partial \delta \theta} &= -[\mathbf{n}_g]_{\times} \frac{\partial \mathbf{n}_\omega}{\partial \delta \theta} = -\frac{1}{r} [\mathbf{n}_g]_{\times} {}^O\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, \\ \frac{\partial \mathbf{r}_{j,4}}{\partial \delta \mathbf{t}} &= -[\mathbf{n}_g]_{\times} \frac{\partial \mathbf{n}_\omega}{\partial \delta \mathbf{t}} = \frac{1}{r} [\mathbf{n}_g]_{\times}, \\ \frac{\partial \mathbf{r}_{j,4}}{\partial \delta \chi_j} &= -[\mathbf{n}_g]_{\times} \frac{\partial \mathbf{n}_\omega}{\partial \delta \chi_j} + [\mathbf{n}_\omega]_{\times} \frac{\partial \mathbf{n}_g}{\partial \delta \chi_j} \\ &= -\frac{1}{r} [\mathbf{n}_g]_{\times} + \frac{1}{\|\mathbf{g}_f\|_2} [\mathbf{n}_\omega]_{\times} (\mathbf{I}_3 - \mathbf{n}_g \mathbf{n}_g^\top) \mathbf{H}_f. \end{aligned} \quad (42)$$

All these Jacobian blocks are assembled into the full matrix $\mathbf{J}_j$ in (??). The overall Jacobian of the manifold cost $J_{\mathcal{M}}$ is then obtained by stacking the contributions from both wheels.

VI. GPU ACCELERATION

The proposed LIO framework contains several computationally intensive modules that are highly amenable to GPU parallelization. Consequently, GPU acceleration can be exploited to improve the real-time performance of the overall system. In particular, two modules benefit the most from GPU implementation:

- 1) **Evaluation of RBF-related terms.** The evaluation of the RBF terrain model involves computing exponential terms for a large number of center–point pairs. This operation is highly parallelizable and is therefore offloaded to the GPU to exploit data-level parallelism. In our implementation, both the matrix–vector multiplication in the terrain height estimation (??) and the entries of the RBF kernel matrix in (??) are computed in parallel on the GPU. In addition, evaluating the manifold soft constraints requires both the residuals in (??) for the left and right wheel contacts and the corresponding Jacobian blocks in (??). These computations repeatedly involve evaluating the RBF terrain function and its spatial derivatives at the contact points. Since each term in the corresponding summations is independent, the contributions in the residual and Jacobian evaluations, including those in (??)–(??), can also be computed in parallel on the GPU.
- 2) **The Sparse kernel matrix inversion.** Although the Gaussian RBF is not compactly supported, its value decays rapidly with distance. Combined with the active-center approximation introduced in Sec. ??, this leads to a numerically sparse kernel matrix $\mathcal{H}_k$ in practice. Therefore, the associated matrix inversion and linear algebra operations can be accelerated using the `cuSOLVER` library from the CUDA Toolkit, which provides efficient sparse linear algebra routines on the GPU. In our implementation, the active-block matrix inversion and the related linear algebra operations appearing in (??), (??), and (??) are all accelerated on the GPU.

VII. EXPERIMENTS

We conduct the experiments on the Limx Tron1 legged-wheel robot in outdoor scenarios. Tron1, developed by Limx Dynamics, features a bipedal lower-body drive with two selectable locomotion modes—*normal* for flat terrain and *off-road* for stairs and uneven ground. In particular, the “off-road locomotion mode” is used in all of the experiments. In addition, the following sensors are available for our experiment (as illustrated in Fig. ??).

- **Mid360 LiDAR & IMU:** mounted on a custom shelf attached to the robot’s body, providing 10 Hz 3D LiDAR point clouds and 200 Hz inertial measurements. As mentioned earlier in Sec. ??, the top-mounted Mid360 LiDAR is installed in an inverted configuration to increase the density of ground observations for terrain reconstruction by directing most beams toward the terrain rather than the sky or distant objects. This inverted mounting is adopted as a sensing configuration for our experiments, rather than as a prerequisite of the proposed method.
- **GPS+RTK:** mounted upright atop the robot, delivering 10 Hz position fixes (latitude, longitude, altitude) of the “ground-truth” trajectory.
- **Joint sensors:** joint angles are measured with a frequency of 500 Hz via the joint sensors.

Moreover, we test our method on a laptop equipped with an Intel i5-12490F processor, 32 GB of RAM, and an NVIDIA GeForce RTX 3070 GPU.

A. The Dataset



Fig. 5. Dataset scenarios: (a) Staircase Scene, (b) Artificial Hill, (c) Rose Garden, (d) Botanical Garden and (e) Indoor Staircase Only. These datasets cover diverse terrain elevations, and each environment contains distinct stair structures for evaluating the z -axis localization performance of different methods.

Due to the lack of publicly available datasets for legged-wheel robots, we collect six datasets in real-world unstructured scenarios that covers a wide variety of indoor and outdoor terrains, surface textures, and elevation profiles. More specifically, these datasets include abrupt elevation changes, mixed feature densities, and both structured and unstructured environments. The scenarios are as follows:

- **Staircase Scene 1 & 2:** These two sequences traverse flat pavements and different staircases with step heights ranging from 5 cm to 15 cm and 1–8 steps, including both single-pass and repeated back-and-forth traversals.
- **Artificial Hill:** This sequence traverses stone-slab paths with multiple single-step stairs, starting from the foot of the hill to the summit, and then returning to the starting point along a different route. It exhibits substantial elevation changes and diverse terrain surface textures.
- **Rose Garden:** This sequence traverses short staircases, flat pavements, and grass slopes, with low lateral obstacles such as flower beds and shrubs. In addition, some staircases are flanked by low walls, which result in sparse geometric features around the staircase region.
- **Botanical Garden:** This sequence covers a long route in the botanical garden, including a suspension bridge, an arch bridge, multiple single-step stairs, and a stone-slab path.
- **Indoor Staircase Only:** This sequence covers a short route that the legged-wheel robot climbs a long indoor stairs. This sequence is mainly used for a stress test for robustness.

Ground Truth: For each sequence (except for “Indoor Staircase Only”), we additionally record an RTK-corrected position trajectory, including latitude, longitude, and altitude, using the GPS+RTK system. This provides centimeter-level 3D reference trajectories for quantitative evaluation. We use the RTK-corrected GPS positions as the ground-truth trajectory. However, since the GPS+RTK system provides only latitude, longitude, and altitude, but not orientation, we will evaluate only the translation errors of the compared methods later on.

B. RBF-based Terrain Approximation Result

In our experiments, we set the mesh resolution of the RBF centers to 0.07 m, the bandwidth parameter σ to 0.04 m, and the measurement noise level σ_ϵ to 0.1. The mesh resolution determines the number of RBF centers and affects both approximation fidelity and computational cost. Empirically, a resolution of 0.07 m provides sufficient geometric detail for typical terrain variations while maintaining real-time efficiency. The bandwidth parameter σ determines the spatial influence of each kernel, while the noise level σ_ϵ accounts for sensor uncertainty and improves robustness to noisy point clouds.

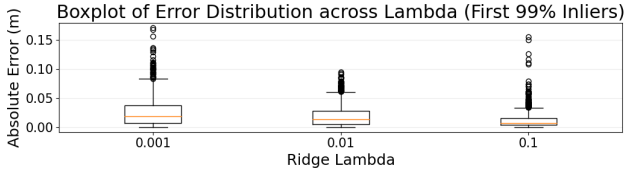


Fig. 6. Boxplots of the terrain-approximation error for different values of λ on the Rose Garden dataset.

In addition, the ridge penalty parameter λ in ridge regression must be chosen carefully. A too small λ may lead to noise-sensitive and numerically unstable fitting, whereas a too large λ may over-smooth the terrain and blur local geometric details. Therefore, λ should be selected to balance numerical stability and terrain-fitting accuracy. To evaluate the effect of λ , we compute the terrain-approximation error using the terrain point cloud observed by the LiDAR. More specifically, at the k -th LiDAR frame, let

$$\mathcal{P}_k^{\text{ter}} = \left\{ \mathbf{p}_k^i = \left[(\mathbf{x}_k^i)^\top, p_{z,k}^i \right]^\top \in \mathbb{R}^3 \right\}_{i=1}^{M_k} \quad (43)$$

denote the local terrain points expressed in the world frame after transforming the LiDAR measurements by the estimated robot pose. Here, $\mathbf{x}_k^i = [p_{x,k}^i, p_{y,k}^i]^\top$ denotes the horizontal coordinate of the i -th terrain point, and $p_{z,k}^i$ denotes its measured height. For each valid terrain point whose horizontal coordinate lies inside the current RBF region of interest, the approximation error is defined as the absolute vertical residual

$$e_k^i = |p_{z,k}^i - f(\mathbf{x}_k^i; \hat{\mathbf{w}}_k^{\text{RG}})|. \quad (44)$$

Therefore, the reported terrain-approximation error measures the consistency between the reconstructed RBF terrain surface and the LiDAR-observed terrain points in the same coordinate frame.

For the parameter study of λ , we collect all valid errors e_k^i over the Rose Garden sequence and compare their distributions under different ridge penalties. Since LiDAR measurements may contain outliers caused by water-puddle reflections, dynamic objects, or local non-terrain returns, we exclude the top 1% largest errors when computing the statistics. As shown in Fig. ??, $\lambda = 0.001$ leads to larger approximation errors because the regression becomes more sensitive to noise and numerical ill-conditioning, while $\lambda = 0.1$ also increases the error due to excessive smoothing. Therefore, we choose $\lambda = 0.01$, which is used in all subsequent experiments.

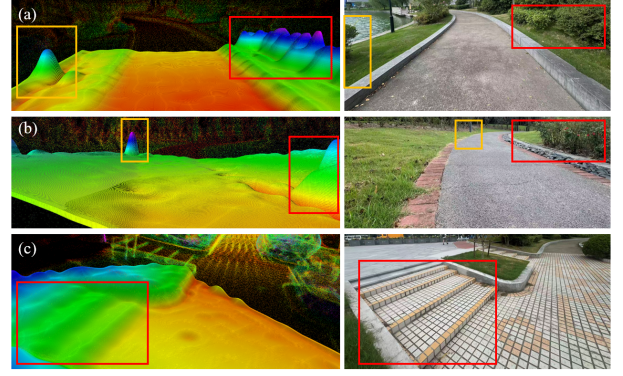


Fig. 7. RBF-based terrain approximation in different environments: (a) flat area, (b) garden pathway, and (c) staircase scene. The colored boxes indicate corresponding objects or local structures across the results.

Next, we qualitatively demonstrate the RBF-based terrain approximation on three representative terrains in Fig. ?? . In the flat-area scene shown in Fig. ??(a), the proposed method reconstructs broad planar regions together with local structures such as bushes and flower beds. In the garden-pathway scene shown in Fig. ??(b), the reconstructed terrain captures the local slope variations, the utility pole on the slope, and the boundary between the pathway and the adjacent grass field. In the staircase scene shown in Fig. ??(c), the method preserves multiple discrete elevation changes corresponding to individual stair steps.

To further quantify the approximation accuracy, we report the error histogram of the RBF-based terrain approximation in the remaining scenarios of the dataset as shown in Fig. ??(d). As a result, under the same evaluation setup, 94.70% of the errors fall below 0.05 m. This indicates that the proposed RBF-based terrain approximation achieves good accuracy on terrains with stairs and grass surfaces. This observation is also consistent with the improved localization performance shown later in Fig. ??, where RBF-LIO reduces the vertical localization drift that is visible in Fast-LIO2.

Moreover, to demonstrate the real-time performance of the proposed method, we report the computation-time histograms of the main modules in Fig. ??(a)–(c). The overall framework runs at 10 Hz, which satisfies the real-time requirement of the LiDAR input stream.

C. Ablation Study

We conduct three ablation studies to evaluate the main components of the proposed framework. The terrain manifold constraint is evaluated on all sequences with RTK ground truth, while the adaptive center selection and GPU acceleration studies are conducted on the Rose Garden sequence, which contains mixed terrain surfaces and representative height variations. First, we disable the terrain manifold constraint to examine whether the RBF-based wheel-terrain residual contributes to pose estimation. Second, we disable the adaptive RBF center selection strategy to evaluate its influence on terrain-fitting accuracy and online computational cost. Third, we compare the GPU and CPU implementations of the RBF

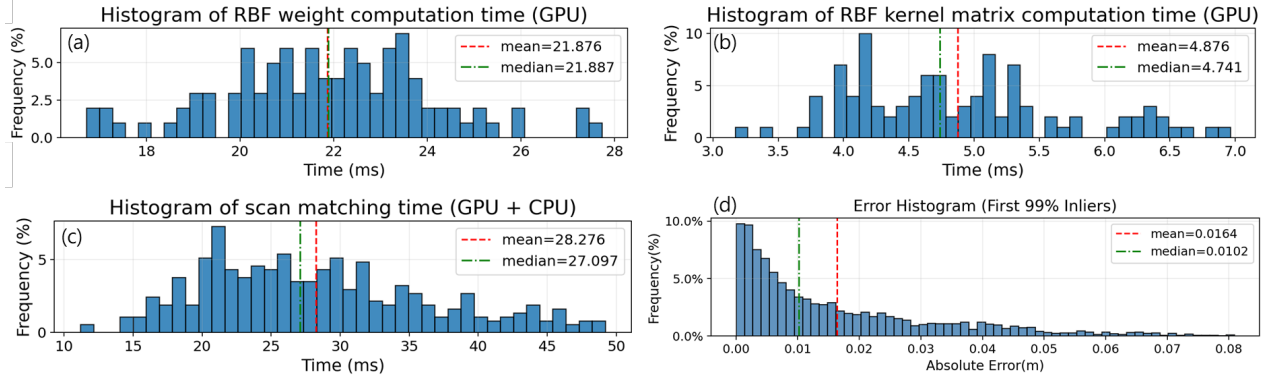


Fig. 8. (a) Histogram of the computation time for RBF weight updates on the GPU. (b) Histogram of the computation time for RBF kernel-matrix evaluation on the GPU. (c) Histogram of the computation time for scan matching on the GPU and CPU. (d) Histogram of the terrain-approximation error on the remaining scenarios in the dataset after excluding the top 1% outliers.

TABLE I

ABLATION STUDY OF THE TERRAIN MANIFOLD CONSTRAINT. VALUES DENOTE ATE RMSE IN METERS, AND THE VALUES IN PARENTHESES DENOTE THE CORRESPONDING z -AXIS ATE RMSE.

Sequence	Full RBF-LIO	w/o manifold	z -ATE reduction
Staircase 1	0.6752 (0.0617)	0.6715 (0.0682)	9.5%
Staircase 2	0.7858 (0.0625)	0.7812 (0.0711)	12.1%
Artificial Hill	0.4759 (0.1354)	0.4771 (0.1859)	27.2%
Rose Garden	0.5824 (0.1034)	0.5844 (0.1112)	7.0%
Botanical Garden	0.9491 (0.2147)	0.9435 (0.2275)	5.6%

terrain approximation module to quantify the benefit of GPU acceleration. In all ablation experiments, only the target component is changed, while the remaining inputs, parameters, and evaluation settings are kept the same whenever applicable.

To verify the contribution of the proposed terrain manifold constraint, we first conduct an ablation study by disabling the manifold residuals in the scan-matching optimization. As shown in Table ??, disabling the terrain manifold constraint has only a limited influence on the overall trajectory-wide ATE. This is expected because the scan-matching term still provides the dominant geometric constraint for horizontal motion, and many portions of the trajectories are locally flat or feature-rich. Nevertheless, the proposed manifold constraint consistently reduces the z -axis ATE in all evaluated sequences. The improvement is most evident on the Artificial Hill sequence, where the robot continuously traverses slopes and single-step structures, and the z -axis ATE is reduced by 27.2%. These results indicate that the terrain manifold constraint mainly acts as a vertical geometric regularizer rather than a replacement for LiDAR scan matching.

Next, we evaluate the adaptive RBF center selection strategy. The full version uses the proposed adaptive center selection strategy, where a candidate RBF center is activated only when it is supported by nearby LiDAR terrain points. The ablated version disables this selection rule and activates all candidate centers inside the current region of interest.

As shown in Table ??, compared with the full version, the average number of active centers increases from 883.8 to 920.9. Since more centers are involved in the fitting process, the mean approximation error is only slightly reduced from

TABLE II

ABLATION STUDY OF THE ADAPTIVE RBF CENTER SELECTION STRATEGY ON THE ROSE GARDEN SEQUENCE. THE RBF TIME IS THE SUM OF KERNEL-MATRIX CONSTRUCTION TIME AND WEIGHT-UPDATE TIME.

Variant	Active centers / ratio	Mean / 95% error	RBF time
Adaptive centers	883.8 / 96.0%	0.0944 / 0.3414	16.88 ms
w/o adaptive centers	920.9 / 100.0%	0.0939 / 0.3399	19.03 ms

TABLE III

ABLATION STUDY OF GPU ACCELERATION FOR ONLINE RBF TERRAIN APPROXIMATION ON THE ROSE GARDEN SEQUENCE. THE TOTAL RBF TIME IS THE SUM OF KERNEL-MATRIX CONSTRUCTION TIME, WEIGHT-SOLVING TIME, AND TERRAIN-EVALUATION TIME.

Variant	Mean / 95% error	Kernel	Weight	Eval.	Total
GPU	0.1176 / 0.4103	2.08 ms	15.53 ms	4.56 ms	22.17 ms
CPU	0.1161 / 0.4057	98.67 ms	4339.21 ms	69.04 ms	4506.92 ms

0.0944 m to 0.0939 m. However, this small accuracy change is obtained at the cost of higher computation. In particular, the average RBF computation time increases from 16.88 ms to 19.03 ms. Equivalently, the proposed adaptive center selection reduces the RBF computation time by about 11.3% while maintaining comparable terrain approximation accuracy.

Finally, we evaluate the contribution of GPU acceleration to the online RBF terrain approximation. The GPU version uses the CUDA-based implementation for kernel-matrix construction, weight solving, and terrain-height evaluation, while the CPU version performs the same operations on the CPU.

As shown in Table ??, the GPU and CPU implementations achieve comparable terrain approximation accuracy. In terms of runtime, the GPU implementation significantly reduces the computational cost of all RBF-related operations. The kernel-matrix construction time decreases from 98.67 ms to 2.08 ms, the weight-solving time decreases from 4339.21 ms to 15.53 ms, and the total RBF computation time decreases from 4506.92 ms to 22.17 ms. This result demonstrates that GPU acceleration is essential for maintaining online terrain reconstruction in the proposed RBF-LIO framework.

TABLE IV

ROBUSTNESS TEST UNDER DEGRADED TERRAIN OBSERVATIONS ON THE ROSE GARDEN SEQUENCE. THE FULL-CLOUD VALIDATION ERROR IS COMPUTED ON THE ORIGINAL FULL TERRAIN POINT CLOUD OF THE SAME FRAME. ERROR VALUES DENOTE MEAN / 95% VERTICAL RESIDUALS IN METERS.

Setting	Kept ratio	Valid frames	Active centers	Full-val. error
Full	100.0%	100.0%	744.6	1.5147 / 4.3010
Sparse 50%	50.0%	100.0%	691.2	1.5492 / 4.3473
Sparse 25%	25.0%	100.0%	616.9	1.4944 / 4.2071
Sector occ.	92.7%	100.0%	658.0	1.5026 / 4.2180

D. Robustness to Degraded Terrain Observations

To examine whether the RBF terrain representation relies heavily on dense ground observations from the inverted LiDAR, we conduct a controlled stress test on the Rose Garden sequence. Before RBF fitting, the terrain point cloud is degraded in four settings: full observations, random sparsification with 50% and 25% retained points, and sector occlusion by removing points within a forward-facing 60° angular sector. The RBF model is fitted using the degraded observations, while the validation error is computed on the original full terrain point cloud of the same frame.

As shown in Table ??, all settings maintain a valid-frame ratio of 100%. Random sparsification reduces the average number of active centers from 744.6 to 691.2 and 616.9, confirming that the terrain support is effectively reduced. Nevertheless, the full-cloud validation errors remain close to the full-observation setting. The sector-occlusion setting shows a similar trend. These results indicate that the RBF terrain approximation remains stable under moderate sparsification and partial field-of-view reduction, as long as sufficient local terrain support is available.

E. Comparison Results

Our proposed RBF-LIO is evaluated against five representative baselines: Fast-LIO2 [?], ROLO-SLAM [?], Leg-KILO [?], GLIM [?], and Wheel-Legged SLAM [?]. In particular, Fast-LIO2 and GLIM have demonstrated strong localization and mapping performance on ground vehicles and aerial platforms. In contrast, ROLO-SLAM, Leg-KILO, and Wheel-Legged SLAM are more closely related to our setting, since they are designed to mitigate z -axis pose drift. Although Leg-KILO and Wheel-Legged SLAM were originally developed for quadruped robots, their core mechanisms of incorporating kinematic constraints into LiDAR-inertial odometry are general. To ensure a fair comparison, we adapt these baselines to our legged-wheel platform. Specifically, we provide the requisite leg-joint kinematics for both Leg-KILO and Wheel-Legged SLAM. In addition, Leg-KILO's contact model is permanently activated to reflect the continuous wheel-terrain contact, while for GLIM, only its odometry module is used. Moreover, at the beginning of each experiment, the robot remains stationary for a period of time to complete state initialization.

To make the comparison protocol explicit, Table ?? summarizes the sensor measurements and additional geometric

information used by each method. We do not modify LiDAR-IMU-only baselines to use joint measurements or terrain contact information, because doing so would change their original algorithmic assumptions and require designing new factors that are outside the scope of those methods. Therefore, Fast-LIO2, GLIM, and ROLO-SLAM are used as representative general-purpose odometry baselines, while Leg-KILO and Wheel-Legged SLAM are used as more closely related kinematic/contact-aware baselines.

In addition, we keep the following settings the same in all comparisons:

- **Dataset and synchronization:** All methods are evaluated on the same rosbag sequences and use the same LiDAR scans recorded by the inverted Mid360 LiDAR(10 Hz). Methods that use IMU measurements are provided with the same Mid360 IMU topic(200 Hz), and methods that use robot kinematics are provided with the same joint-sensor topic(500 Hz).
- **Initialization:** At the beginning of each experiment, the robot remains stationary for a short period to allow state initialization. The same initialization segment is used for all methods.
- **Preprocessing and resolution:** The same voxel resolution of 0.2 m is used for LiDAR down-sampling in scan matching. The same LiDAR frame range and evaluation interval are used when computing the trajectory errors.
- **IMU parameters:** For the methods that use IMU measurements, the noise covariance matrices for linear acceleration and angular velocity are set to $10^{-2}I_3$, while the corresponding bias covariance matrices are set to $10^{-4}I_3$.
- **Evaluation metric:** Since the GPS+RTK system provides only position measurements without orientation, all methods are evaluated using translation errors only. The same RTK trajectory, time association, and alignment protocol are used for all quantitative comparisons.

We first examine a staircase-only indoor scenario to highlight the robustness of the compared methods under abrupt elevation changes. As shown in Fig. ??, RBF-LIO produces a structurally consistent point cloud map of the staircase. In contrast, the maps generated by the other methods exhibit varying degrees of ghosting and layering artifacts, suggesting reduced robustness under repeated abrupt elevation changes. This experiment serves as a stress test and directly illustrates the advantage of explicitly incorporating terrain geometry in environments dominated by abrupt height transitions.

We further qualitatively evaluate the compared methods on our collected datasets. The improvements are indeed most visible along the vertical dimension. As illustrated in Fig. ??, Fast-LIO2 suffers from pose drift and map layering artifacts in Rose Garden, where sparse features and abrupt elevation changes occur simultaneously. Moreover, on the same staircase in Rose Garden, Wheel-Legged SLAM incorrectly fits the plane and fails to map. In addition, in the Botanical Garden dataset, where the terrain is more complex, Leg-KILO suffers from pose drift, which is consistent with the limitation of its coarse terrain model. Although kinematic constraints can improve estimation in feature-sparse environments, the local

TABLE V
SENSOR MEASUREMENTS AND ADDITIONAL GEOMETRIC INFORMATION USED BY EACH METHOD IN THE COMPARISON.

Method	LiDAR	IMU	Joint sensors	Kinematic/contact information	Explicit terrain model
Fast-LIO2 [1]	Yes	Yes	No	No	No
GLIM [2]	Yes	No	No	No	No
ROLO-SLAM [4]	Yes	No	No	Rotation constraint	No
Leg-KILO [3]	Yes	Yes	Yes	Leg kinematics and local contact plane	Local planar model
Wheel-Legged SLAM [5]	Yes	Yes	Yes	Wheel-leg kinematics	Local contact constraint
RBF-LIO (ours)	Yes	Yes	Yes	Wheel-terrain contact geometry	Online RBF terrain manifold

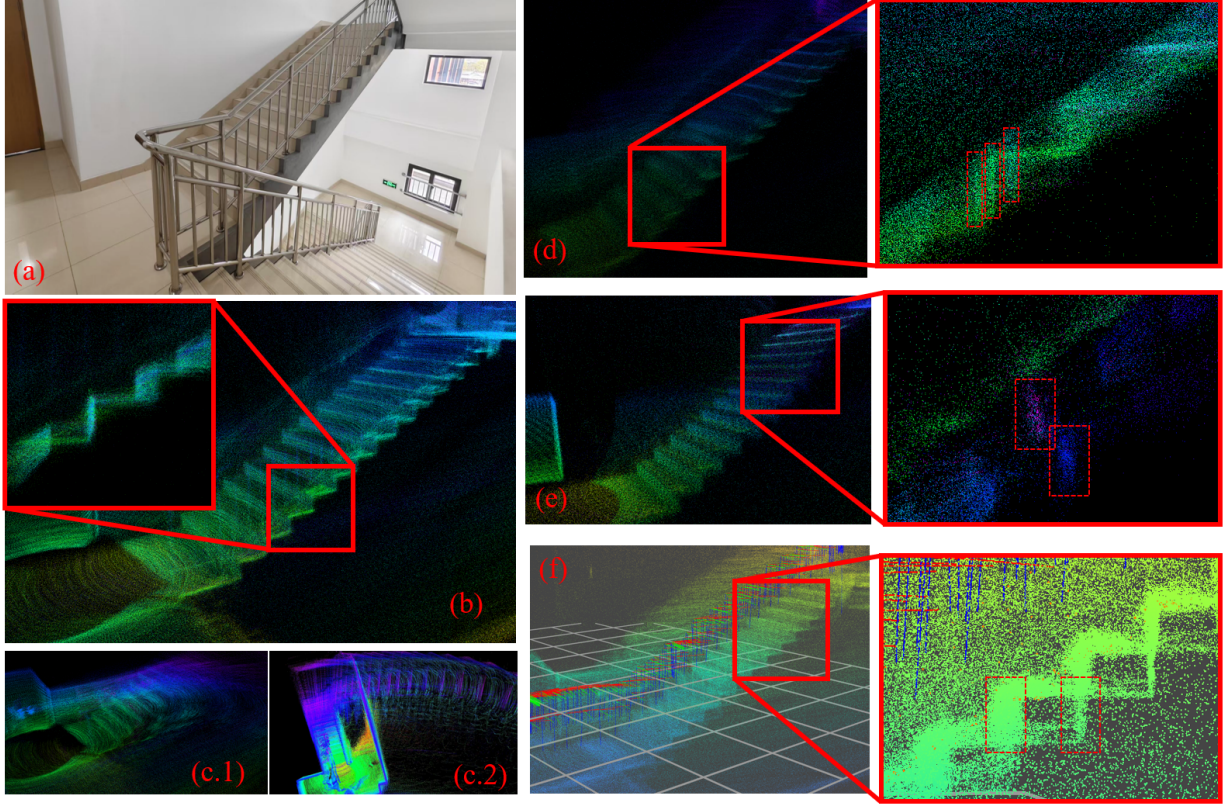


Fig. 9. Qualitative comparison of point cloud maps generated in an indoor staircase-only scenario. (a) Reference photograph of the environment. Mapping results produced by (b) RBF-LIO, (c.1) Wheel-Legged SLAM, (c.2) top view of Wheel-Legged SLAM, (d) FAST-LIO2, (e) Leg-KILO, and (f) GLIM. The zoomed-in views highlight the staircase structure reconstructed by each method. The dashed red rectangles indicate the same vertical plane of a single step for visual comparison.

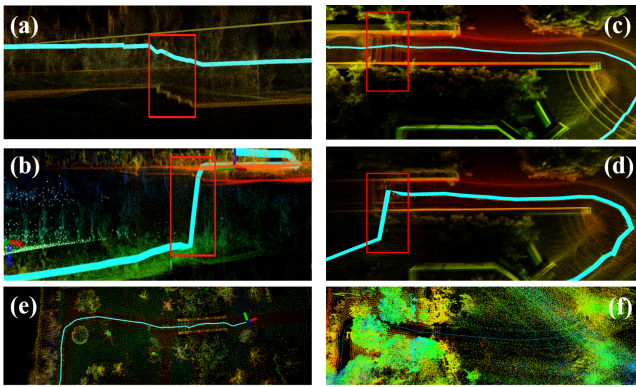


Fig. 10. Visualization of the mapping results: (a) RBF-LIO on the Rose Garden dataset, (b) Fast-LIO2 on the Rose Garden dataset, (c) RBF-LIO on a staircase in a feature-rich environment, (d) Wheel-Legged SLAM in the same environment as (c), (e) RBF-LIO on the Botanical Garden dataset, and (f) Leg-KILO on the Botanical Garden dataset.

flatness assumption in Leg-KILO is likely to be violated by uneven structures such as the suspension bridge and the arch bridge, while the reduced geometric richness may further degrade plane extraction. In contrast, RBF-LIO continuously corrects the height estimate through the manifold soft constraints, thereby preserving a more globally consistent map.

Furthermore, we compare the trajectories in Rose Garden. We choose this scenario as a representative one because it contains sparse features together with abrupt height changes. In particular, Fast-LIO2 and Wheel-Legged SLAM both diverge, while the pose estimation error of ROLO-SLAM is too large. Therefore, the trajectories of these three methods are not depicted in Fig. ???. As shown in Fig. ??, Leg-KILO and GLIM exhibit noticeable z-axis drift, whereas RBF-LIO maintains the closest alignment with the RTK reference.

Next, we quantitatively compare the performance of RBF-

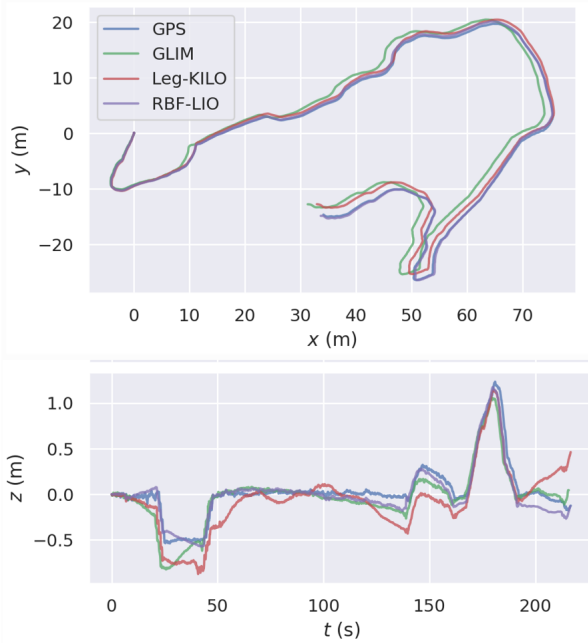


Fig. 11. Trajectories of different methods on the Rose Garden dataset: projection on the xy -plane and evolution along the z -axis.

LIO with the baselines. In particular, we report the Absolute Trajectory Error (ATE) and Relative Trajectory Error (RTE) of all methods in Table. ?? and Table. ?. In addition to the overall trajectory errors, we also report the errors of the z component in parentheses to directly evaluate the effect of the proposed manifold soft constraints on vertical localization. The best results are highlighted in bold font.

Both GLIM and RBF-LIO succeed in estimating the poses on all datasets. In scenarios with rich geometric features but abrupt height changes, such as Artificial Hill and Staircase Scene 1 & 2, the ATE of RBF-LIO is generally comparable to that of the strongest competing methods. This suggests that, in feature-rich environments, the baselines can maintain comparable overall trajectory accuracy. However, as the terrain becomes more complex, the difference between the methods becomes more evident. Moreover, RBF-LIO also outperforms ROLO-SLAM, which is explicitly designed to reduce vertical errors.

Notably, the improvement in RTE is less pronounced than that in ATE. This is consistent with the different roles of the two metrics: RTE evaluates short-horizon local consistency over a fixed interval, whereas ATE reflects the accumulated global drift over the entire trajectory. Since the proposed manifold constraints mainly take effect when the robot traverses abrupt elevation transitions such as stairs, their benefit is diluted in trajectory-wide RTE statistics that average over many locally easy segments. In contrast, the suppression of vertical drift accumulates over time and is therefore more clearly reflected in ATE, especially in the z -axis error.

To summarize, both the quantitative and qualitative results show that the proposed RBF-LIO offers a more robust performance in scenarios that have abrupt elevation changes than the baselines. In particular, the advantage of RBF-LIO is most

evident in its suppression of accumulated vertical drift and its ability to preserve global consistency in challenging terrain, yet still performs comparably well on normal terrain.

F. Wet-Grass Stress Test

To evaluate the behavior of RBF-LIO under slippery-surface and contact-uncertain conditions, we collect an additional wet-grass sequence after rain using the same sensor configuration as the other outdoor sequences. We compare the full RBF-LIO with Fast-LIO2, GLIM, and RBF-LIO without the terrain manifold constraint.



Fig. 12. Wet-grass scenario.

As shown in Table ??, all methods complete the sequence without divergence. RBF-LIO achieves the lowest ATE and z -ATE. Compared with the ablated version without the terrain manifold constraint, the full method reduces the z -ATE from 0.1850 m to 0.1442 m, corresponding to a relative reduction of about 22.1%. The RTE values of the full and ablated variants are close, indicating that the manifold constraint improves vertical consistency without degrading short-term odometry stability.

This stress test also illustrates the limitation of contact-based terrain constraints. On slippery or deformable surfaces, wheel-terrain contact may become unreliable. Therefore, the manifold term is used as a soft residual rather than a hard constraint. When large manifold residuals, insufficient terrain support, abnormal RBF fitting errors, or sudden IMU changes are detected, the constraint can be down-weighted or disabled, allowing the estimator to fall back to LiDAR-IMU scan matching.

G. Cross-Platform Staircase Stress Test

We further conduct a qualitative cross-platform staircase test on a DeepRobotics Lynx M20 four-wheel-legged robot. Unlike the Tron1 platform used in the main experiments, the M20 uses a front-facing LiDAR configuration, which provides less favorable and less uniform ground observations than the inverted Mid360 setup. The selected half-floor staircase segment contains 183 poses over 36.40 s, with a trajectory length of 12.03 m and an elevation range of 2.23 m.

As shown in Fig. ??, RBF-LIO generates a continuous staircase map on the M20 platform. Compared with the Tron1 results, the reconstructed map contains more local noise and thickness, which is mainly caused by the front-facing LiDAR and sparser ground observations. Nevertheless,

TABLE VI

ATE [?] (M) FOR DIFFERENT METHODS. VALUES IN PARENTHESES DENOTE z -AXIS ERRORS. SLASH MEANS FAILURE.

Sequence	RBF-LIO		Fast-LIO2 [?]		ROLO-SLAM [?]		Leg-KILO [?]		GLIM [?]		Wheel-Legged SLAM [?]	
	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max
Staircase 1	0.6752 (0.0617)	1.3126 (0.3588)	0.7471 (0.0587)	1.4464 (0.3166)	21.6712 (1.0336)	29.2766 (2.4170)	0.6347 (0.0615)	1.1855 (0.3393)	0.7357 (0.1793)	1.3265 (0.4787)	1.4982 (0.3200)	2.5845 (0.7898)
Staircase 2	0.7858 (0.0625)	1.2764 (0.3372)	0.9039 (0.0670)	1.5034 (0.3207)	36.7933 (3.8083)	65.8538 (9.8349)	0.8160 (0.0832)	1.5232 (0.3841)	0.9872 (0.2074)	1.7557 (0.5310)	1.5265 (0.2032)	3.6606 (0.5622)
Artificial Hill	0.4759 (0.1354)	0.9555 (0.5144)	0.6088 (0.1553)	1.1251 (0.5210)	16.1785 (6.4400)	24.1648 (11.2143)	0.4534 (0.1394)	0.8362 (0.5162)	0.6190 (0.2300)	1.1597 (0.6370)	1.8872 (0.3987)	3.3049 (1.0807)
Rose Garden	0.5824 (0.1034)	1.0972 (0.2628)	/	/	16.9995 (1.5915)	27.4158 (4.8372)	0.9069 (0.1669)	1.8861 (0.6708)	1.2513 (0.0965)	2.1290 (0.3550)	/	/
Botanical Garden	0.9491 (0.2147)	1.6648 (0.4915)	1.2962 (0.3683)	4.0772 (1.2227)	22.3889 (3.3272)	41.6459 (7.2956)	/	/	0.9980 (0.1595)	1.8093 (0.4791)	2.6237 (0.3509)	4.5260 (0.8286)

TABLE VII

RTE [?] (M) ($\Delta t = 0.1$ s) FOR DIFFERENT METHODS. VALUES IN PARENTHESES DENOTE z -AXIS ERRORS. SLASH MEANS FAILURE.

Sequence	RBF-LIO		Fast-LIO2 [?]		ROLO-SLAM [?]		Leg-KILO [?]		GLIM [?]		Wheel-Legged SLAM [?]	
	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max
Staircase 1	0.0503 (0.0170)	0.2267 (0.1333)	0.0556 (0.0124)	0.2546 (0.1058)	0.1694 (0.0401)	1.1933 (0.3205)	0.0478 (0.0124)	0.2344 (0.1064)	0.0545 (0.0232)	0.4161 (0.4037)	0.1073 (0.0108)	0.7914 (0.0968)
Staircase 2	0.0493 (0.0138)	0.2227 (0.0758)	0.0493 (0.0148)	0.2333 (0.1006)	0.2027 (0.0650)	0.8776 (0.3859)	0.0461 (0.0141)	0.2294 (0.1095)	0.0480 (0.0140)	0.2257 (0.1025)	0.0869 (0.0113)	0.8077 (0.0779)
Artificial Hill	0.0277 (0.0314)	0.4614 (0.4198)	0.0864 (0.0365)	0.5699 (0.4227)	0.3977 (0.0993)	8.7865 (0.7038)	0.0605 (0.0301)	0.4617 (0.4434)	0.0624 (0.0305)	0.4645 (0.4478)	0.1233 (0.0336)	1.1333 (0.4536)
Rose Garden	0.0363 (0.0154)	0.1744 (0.0979)	/	/	0.1881 (0.0530)	0.6960 (0.2968)	0.0595 (0.0114)	0.2337 (0.0860)	0.0413 (0.0113)	0.2288 (0.1415)	/	/
Botanical Garden	0.0849 (0.0179)	0.3815 (0.3343)	0.0664 (0.0197)	0.5346 (0.3514)	0.2104 (0.0639)	0.8683 (0.3172)	/	/	0.0575 (0.0185)	0.3550 (0.3472)	0.0938 (0.0168)	0.7006 (0.3530)

TABLE VIII

WET-GRASS STRESS TEST UNDER A SLIPPERY-SURFACE CONDITION.
ATE, z -ATE, AND RTE ARE REPORTED IN METERS.

Method	ATE	z -ATE	RTE
Fast-LIO2	0.8836	0.2248	0.068
GLIM	0.7911	0.1712	0.059
RBF-LIO w/o manifold	0.7654	0.1850	0.054
RBF-LIO	0.7023	0.1442	0.055

the staircase structure remains recognizable and the mapping process does not diverge. In contrast, Leg-KILO produces a blurred and thick staircase map, while Wheel-Legged SLAM fails to preserve a consistent staircase structure and exhibits severe trajectory drift. This provides qualitative evidence that the LiDAR-inertial and terrain-aware mapping pipeline can operate on a different wheel-legged platform and LiDAR mounting configuration.

Since the M20 platform differs from Tron1 in morphology, LiDAR mounting, ground-observation density, and wheel-terrain contact geometry, a complete transfer of the full manifold constraint requires platform-specific kinematic parameters and contact-reliability checks. More generally, the proposed manifold constraint assumes reliable wheel-terrain contact, sufficient local terrain support, and a terrain surface that can be locally represented as a single-valued height function. These assumptions may be weakened near sharp stair edges, highly deformable grass, severe wheel slip, temporary loss of contact, strong vibration, or multi-layer structures. In such

cases, the manifold residual should be down-weighted or rejected, allowing the estimator to rely on the LiDAR-IMU scan-matching objective.

VIII. CONCLUSION

In this paper, we proposed RBF-LIO, a LiDAR-inertial odometry framework that incorporates terrain constraints to improve vertical localization accuracy. In particular, the terrain is approximated by radial basis functions (RBFs) based on moment conditions, and the resulting estimation problem is further reformulated as a recursive ridge regression problem. This yields a smooth terrain manifold, which is then introduced into the gradient-based LIO optimization as a set of soft constraints to mitigate localization errors along the z -axis. Experimental results demonstrate that the proposed method significantly suppresses z -axis pose drift in environments with large height differences and abrupt elevation changes, compared with state-of-the-art methods. As a result, the introduced soft constraints enable continuous height correction in the LIO framework, thereby producing coherent and high-fidelity maps even in staircase scenarios and feature-sparse environments. Moreover, we release a dataset to support future SLAM research on legged-wheel robots.

REFERENCES

- [1] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lid2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.

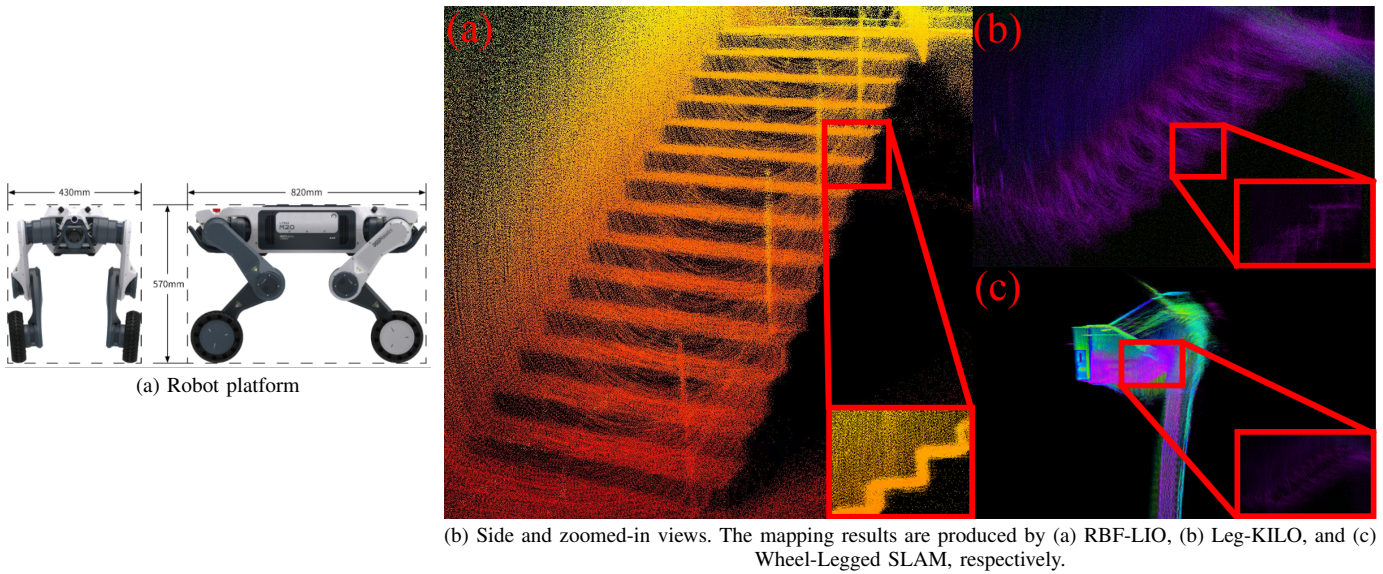


Fig. 13. Qualitative cross-platform staircase stress test on the M20 four-wheel-legged robot. The LiDAR is mounted in a front-facing configuration, which provides less favorable ground observations than the inverted Mid360 setup used on the Tron1 platform. In the zoomed-in comparison, RBF-LIO preserves a clearer staircase structure, Leg-KILO produces a blurred and thick map, and Wheel-Legged SLAM suffers from severe drift.

- [2] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Glim: 3d range-inertial localization and mapping with gpu-accelerated scan matching factors," *Robotics and Autonomous Systems*, vol. 179, p. 104750, 2024.
- [3] G. Ou, D. Li, and H. Li, "Leg-kilo: Robust kinematic-inertial-lidar odometry for dynamic legged robots," *IEEE Robotics and Automation Letters*, 2024.
- [4] Y. Wang, B. Ren, X. Zhang, P. Wang, C. Wang, R. Song, Y. Li, and M. Q.-H. Meng, "Rolo-slam: rotation-optimized lidar-only slam in uneven terrain with ground vehicle," *Journal of Field Robotics*, vol. 42, no. 3, pp. 880–902, 2025.
- [5] Y. Wen, Y. Li, Q. Shang, C. Jiang, H. Hou, H. Liu, Y. Zhang, and L. Han, "Wheel-legged slam: Indoor lidar-inertial slam integrating kinematic model of wheel-legged robots," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1273–1280, 2024.
- [6] J. Zhang, S. Singh *et al.*, "Loam: Lidar odometry and mapping in real-time," in *Robotics: Science and systems*, vol. 2, no. 9. Berkeley, CA, 2014, pp. 1–9.
- [7] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2020, pp. 5135–5142.
- [8] W. Xu and F. Zhang, "Fast-lid: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.
- [9] D. Wisht, M. Camurri, and M. Fallon, "Vilens: Visual, inertial, lidar, and leg odometry for all-terrain legged robots," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 309–326, 2022.
- [10] Q. Zhao, M. Li, Y. Shi, X. Chen, Z. Yu, L. Han, Z. Fu, J. Zhang, C. Li, Y. Zhang *et al.*, "Liko: Lidar, inertial, and kinematic odometry for bipedal robots," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 1180–1185.
- [11] J. Li, C. Li, H. Chen, H. Guo, X. Luo, C. P. Chen, and C. Yang, "Ars-slam: Accurate robust spinning lidar slam for a quadruped robot in large-scale scenario," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 2635–2641.
- [12] C. Zhang, Z. Yang, Q. Fang, C. Xu, H. Xu, X. Xu, and J. Zhang, "Frl-slam: A fast, robust and lightweight slam system for quadruped robot navigation," in *2021 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2021, pp. 1165–1170.
- [13] P. Fankhauser, M. Bloesch, C. Gehring, M. Hutter, and R. Siegwart, "Robot-centric elevation mapping with uncertainty estimates," in *International Conference on Climbing and Walking Robots (CLAWAR)*, 2014.
- [14] P. Fankhauser, M. Bloesch, and M. Hutter, "Probabilistic terrain mapping for mobile robots with uncertain localization," *IEEE Robotics and Automation Letters (RA-L)*, vol. 3, no. 4, pp. 3019–3026, 2018.
- [15] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, "Elevation mapping for locomotion and navigation using gpu," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 2273–2280.
- [16] W. Tian, J. Wang, P. Yang, W. Xiao, and S. Zlatanova, "Lio-gc: Lidar inertial odometry with adaptive ground constraints," *Remote Sensing*, vol. 17, no. 14, p. 2376, 2025.
- [17] A. Leininger, M. Ali, H. Jardali, and L. Liu, "Gaussian process-based traversability analysis for terrain mapless navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 10925–10931.
- [18] J. C. Carr, R. K. Beatson, J. B. Cherrie, T. J. Mitchell, W. R. Fright, B. C. McCallum, and T. R. Evans, "Reconstruction and representation of 3d objects with radial basis functions," in *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, 2001, pp. 67–76.
- [19] C. Chen, Y. Li, N. Zhao, B. Guo, and N. Mou, "Least squares compactly supported radial basis function for digital terrain model interpolation from airborne lidar point clouds," *Remote Sensing*, vol. 10, no. 4, p. 587, 2018.
- [20] D. G. Luenberger, *Optimization by vector space methods*. John Wiley & Sons, 1997.
- [21] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [22] J. J. Moré, "The levenberg-marquardt algorithm: implementation and theory," in *Numerical analysis: proceedings of the biennial Conference held at Dundee, June 28–July 1, 1977*. Springer, 2006, pp. 105–116.
- [23] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual (-inertial) odometry," in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2018, pp. 7244–7251.